

claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf_2be29ecf-a31\agent-adeb9c83b07ffbe54.jsonl`  
- SHA-256: `a2e746f829925c6165cb375937ad840eab5a76110cf63ad36e0415c3d49ecc14`  
- Source modified: `2026-06-22T01:28:48+00:00`  
- Imported at: `2026-07-05T16:48:23+00:00`  
- project: `wf_2be29ecf-a31`  
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`
```

Transcript

1. user (2026-06-22T01:26:03.394Z)

You are reviewing PR #310 "P2b-install" for CORRECTNESS + EDGE CASES of the ENGINE change. Run `cd C:\Users\avidu\Projects\bhi-p2b && git diff origin/master...HEAD` and read the touched engine files in `C:\Users\avidu\Projects\bhi-p2b: backend/config/loader.py (write_light_default_config + LIGHT_DEFAULT_CONFIG), backend/config/__init__.py, backend/main.py (the if args.app: seeding block before load_app_config)`. Scrutinize: (1) Is the seeding placed correctly ? BEFORE `load_app_config` so the loader reads it, and gated ONLY on `args.app` (so `--server / CLI` single-shot stay byte-identical)? (2) Does `write_light_default_config` truly never overwrite (idempotent) and create parent dirs? (3) Does the MINIMAL seeded config actually load via the `loader+truly-local` preset to `default_segmenter=motion + skip_ai_handoff=true + storage=local + use_gpu=false` WITHOUT tripping a startup ERROR (motion needs no torch/GPU; server passes `cuda_available=None`)? (4) Any interaction bug with `--output-dir / app_home` resolution / the P0e single-instance lock? (5) Is 'motion' a real registered torch-free segmenter? Report concrete findings only (file:line, why it's a bug, fix). If you find nothing real, return an empty findings list.

2. assistant (2026-06-22T01:26:06.190Z)

I'll review PR #310. Let me start by examining the diff and the touched files.

3. user (2026-06-22T01:26:09.360Z)

```
78e423d docs(pkg): link P2b-install PR #310 in the $7 tracker  
---BRANCH---  
feat/pkg-p2b-install
```

4. assistant (2026-06-22T01:26:12.069Z)

Let me look at the diff and touched files.

5. user (2026-06-22T01:26:12.739Z)

```
README.md | 31 ++++++  
backend/config/__init__.py | 9 +-  
backend/config/loader.py | 34 ++++++  
backend/main.py | 15 ++++  
docs/PACKAGING_PLAN.md | 13 +-  
scripts/install.py | 210 ++++++  
scripts/uninstall.py | 150 ++++++  
tests/test_app_launcher.py | 42 ++++++  
tests/test_p2b_install.py | 163 ++++++  
9 files changed, 660 insertions(+), 7 deletions(-)
```

6. user (2026-06-22T01:26:15.688Z)

```
diff --git a/backend/config/__init__.py b/backend/config/__init__.py
index 9b276c0..6c38999 100644
--- a/backend/config/__init__.py
+++ b/backend/config/__init__.py
@@ -6,7 +6,12 @@ Public API:
 This package is the canonical place for all configuration access.
 """

-from .loader import load_config, save_default_config
+from .loader import (
+    LIGHT_DEFAULT_CONFIG,
+    load_config,
+    save_default_config,
+    write_light_default_config,
+)
 from .models import ( # re-export common ones
     AppConfig,
     Config,
@@ -27,6 +32,8 @@ from .startup import (
 __all__ = [
     "load_config",
     "save_default_config",
+    "write_light_default_config",
+    "LIGHT_DEFAULT_CONFIG",
     "AppConfig",
     "Config",
     "DeploymentConfig",
diff --git a/backend/config/loader.py b/backend/config/loader.py
index 37ed101..274c4f7 100644
--- a/backend/config/loader.py
+++ b/backend/config/loader.py
@@ -212,6 +212,40 @@ def save_default_config(path: str | Path | None = None) -> Path:
     return cfg_path

+# The shipped batteries-included LIGHT-tier default (PACKAGING_PLAN.md P2b / gate 08).
+# Deliberately MINIMAL: the `truly-local` PROFILE preset (presets.py) supplies skip_ai_handoff,
+# local storage and the compute_target, so this stays the handful of knobs a fresh, non-technical
+# user gets ? a torch-free, offline, zero-setup experience:
+# * default_segmenter "motion" ? pure-OpenCV, no torch / GPU / weights / API key (LIGHT tier);
+# * profile "truly-local" ? no Gemini handoff, local storage ? never surprise-bills on Gemini nor returns 0 rallies on WASB-without-a-key (08 intent);
+# * use_gpu false ? honest for a LIGHT (no-GPU) box (motion ignores it either way).
+# The first-run wizard (P2c) upgrades this to Gemini once the user supplies a key.
+LIGHT_DEFAULT_CONFIG: dict[str, Any] = {
+    "sport": "badminton",
+    "deployment": {"profile": "truly-local"},
+    "indexing": {"default_segmenter": "motion", "use_gpu": False},
+}
+
+def write_light_default_config(
+    path: str | Path | None = None, *, overwrite: bool = False
+) -> tuple[Path, bool]:
+    """Seed the batteries-included LIGHT default config (`LIGHT_DEFAULT_CONFIG`).
+
+    Returns `(path, written)`. Unlike :func:`save_default_config` (which dumps a FULL AppConfig and overwrites), this writes the minimal profile-driven LIGHT bundle and ? by default ? **never overwrites an existing file**, so a user's edits or the first-run wizard's choices are
```

```

preserved
+ (idempotent first-run seeding). Pass ``overwrite=True`` to force. Creates parent dirs.
+ """
+ cfg_path = Path(path) if path is not None else default_config_path()
+ if cfg_path.exists() and not overwrite:
+     return cfg_path, False
+ cfg_path.parent.mkdir(parents=True, exist_ok=True)
+ cfg_path.write_text(json.dumps(LIGHT_DEFAULT_CONFIG, indent=2) + "\n", encoding="utf-8")
+ return cfg_path, True
+
+
# Backwards-compatible alias used by some older call sites during transition.
def get_default_config() -> dict[str, Any]:
    """Return defaults as a plain dict (for transitional use only)."""
diff --git a/backend/main.py b/backend/main.py
index c1738b4..4e5a19a 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -48,6 +48,7 @@ from backend.utils.single_instance import acquire_lock # P0e: one instance
per
from backend.utils.ffmpeg import ffmpeg_bin, ffprobe_bin # P2a: single ffmpeg/ffprobe resolver
from backend.config import ( # new typed config
    load_config as load_app_config,
+ write_light_default_config, # P2b: batteries-included first-run config seeding
    AppConfig,
    StitchingConfig,
    enforce_startup_checks,
@@ -934,6 +935,20 @@ def main():
    run_cli_diagnose_clip(args.video_path, args.debug_clip)
    return

+ # P2b (PACKAGING_PLAN.md gate 08): batteries-included first run for the friendly `--app`
+ # launcher. Seed a truly-local, torch-free LIGHT default config (detector 'motion' ?
+ offline,
+ # no API key, no GPU, no cloud bills) at the resolved app-home (default_config_path(),
+ which
+ # roots under RALLY_APP_HOME when the packaged shortcut pins it) BEFORE load_app_config
+ reads
+ # it ? so a fresh `pip`/`uv` tool install + `indexer --app` just works. NEVER overwrites an
+ # existing config (a user's edits / the P2c wizard's choices are preserved). Scoped to
+ --app
+ # (the packaged app mode); `--server` and the CLI single-shot are byte-identical (no
+ seeding).
+ if args.app:
+     seeded_path, seeded = write_light_default_config()
+     if seeded:
+         print(f"[APP] First run: seeded a truly-local default config at {seeded_path} "
+             f"(detector 'motion' ? offline, no API key or GPU needed). "
+             f"Use the in-app setup to switch to Gemini.")
+
+ # Initialize Global Config and DB
+ global global_config, db_instance
+ global_config = load_app_config() # returns typed AppConfig

```

7. assistant (2026-06-22T01:26:18.262Z)

Now let me read the surrounding context of main.py to understand the ordering relative to load_app_config, the single-instance lock, and --output-dir handling.

8. user (2026-06-22T01:26:19.286Z)

```

860
861     def main():
862         parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")

```

```

863     parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
864     parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
865     parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
866     parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
867     parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
868     parser.add_argument("--app", action="store_true",
869                         help="Launch the app: start the local server (localhost-only)
and open the "
870                             "bundled UI in your browser. Falls back to a free port if
--port is busy.")
871     parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
872     parser.add_argument("--host", default=None,
873                         help="Bind address. Default 127.0.0.1 (localhost-only); 0.0.0.0
when "
874                             "security.edge_auth_enabled. Expose on a network ONLY
behind an auth gate.")
875     parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")
876
877     available_segmenters = list(segmenter_registry.list_available().keys())
878     parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
879     parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
880     parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")
881     parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")
882
883     args = parser.parse_args()
884
885     if args.setup:
886         # Invoke the diagnostics/bootstrap script (renamed from the old root
887         # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
888         # build system). Resolve its path from this file so `indexer --setup`
889         # works regardless of the caller's cwd.
890         import subprocess
891         doctor_path = os.path.join(
892             os.path.dirname(os.path.abspath(__file__)),
893             "scripts", "doctor.py",
894         )
895         print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
896         subprocess.run([sys.executable, doctor_path])
897         return
898
899     if args.trim_start is not None and args.trim_end is not None:
900         if not args.video_path:
901             print("[ERROR] Please provide --video-path to extract a segment.")
902             return
903
904         # Determine output path
905         out_filename = args.trim_output or "trimmed_segment.mp4"
906         if os.path.isabs(out_filename):
907             out_path = out_filename
908             out_dir = os.path.dirname(out_path)
909             out_name = os.path.basename(out_path)
910         else:
911             out_dir = resolve_output_dir(args.output_dir) # P0a: app-home aware (CWD-
identical when unset)
912             out_path = os.path.join(out_dir, out_filename)

```

```

913         out_name = out_filename
914
915         os.makedirs(out_dir, exist_ok=True)
916         print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")
917
918         # global_config isn't initialized this early; load the typed config so the trim
919         # clip carries the same configured stitching paddings + watermark as a real
920         # highlight (watermark default-on).
921         stitching = load_app_config().stitching
922         compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
923                                 end_padding=stitching.end_padding,
watermark=stitching.watermark)
924         try:
925             res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
926             print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
927         except Exception as e:
928             print(f"[ERROR] Failed to compile trimmed segment: {e}")
929         return
930
931     if args.debug_clip is not None:
932         if not args.video_path:
933             print("[ERROR] Please provide --video-path to debug a specific clip.")
934             return
935         run_cli_diagnose_clip(args.video_path, args.debug_clip)
936         return
937
938     # P2b (PACKAGING_PLAN.md gate 08): batteries-included first run for the friendly
`--app`
939     # launcher. Seed a truly-local, torch-free LIGHT default config (detector 'motion' ?
offline,
940     # no API key, no GPU, no cloud bills) at the resolved app-home
(default_config_path(), which
941     # roots under RALLY_APP_HOME when the packaged shortcut pins it) BEFORE
load_app_config reads
942     # it ? so a fresh `pip`/`uv` tool install + `indexer --app` just works. NEVER
overwrites an
943     # existing config (a user's edits / the P2c wizard's choices are preserved). Scoped
to --app
944     # (the packaged app mode); `--server` and the CLI single-shot are byte-identical (no
seeding).
945     if args.app:
946         seeded_path, seeded = write_light_default_config()
947         if seeded:
948             print(f"[APP] First run: seeded a truly-local default config at
{seeded_path} "
949                 f"(detector 'motion' ? offline, no API key or GPU needed). "
950                 f"Use the in-app setup to switch to Gemini.")
951
952     # Initialize Global Config and DB
953     global global_config, db_instance
954     global_config = load_app_config() # returns typed AppConfig
955     _enforce_startup_or_exit(global_config) # M5: fail fast on an incoherent ACTIVE
profile
956
957     # The CLI --output-dir overrides the configured output directory. It now lives
958     # on the typed model (OutputConfig.output_dir), so every consumer ? including
959     # /api/compile ? reads the same value instead of a write-only runtime hack.
960     # P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME); with the env
961     # unset, resolve_output_dir("output") == "output" (CWD-relative) ? byte-identical to
before.
962     global_config.output.output_dir = resolve_output_dir(args.output_dir)
963     os.makedirs(global_config.output.output_dir, exist_ok=True)
964

```

```

965         db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
966
967         # P0e: refuse a SECOND instance against the same DB BEFORE the destructive stale-job
sweep below.
968         # The job worker is in-process, so a concurrent process opening this DB would sweep
THIS one's
969         # in-flight jobs to 'failed'. The OS lock (held for the process lifetime; auto-
released on exit
970         # or crash) is keyed to db_path, so a different --output-dir/DB still runs
concurrently.
971         global _instance_lock
972         try:
973             _instance_lock = acquire_lock(db_path + ".lock")
974         except OSError as exc:
975             # The lock path isn't writable (read-only dir / perms). Don't crash and don't
falsely claim a
976             # second instance ? warn and proceed WITHOUT the guard (a lock-infra failure
must not block a
977             # legitimate single instance).
978             logger.warning("[JOBS] Could not create the single-instance lock at %s.lock
(%s); proceeding "
979                           "WITHOUT the second-instance guard.", db_path, exc)

```

9. assistant (2026-06-22T01:26:22.527Z)

The seeding is placed before ``load_app_config()`` and gated on ``args.app`` ? correct ordering. Now let me verify the loader: ``default_config_path``, ``load_config``, the ``truly-local`` preset, and how ``default_segmenter`` / profile interact. Let me read the loader fully.

10. user (2026-06-22T01:26:23.523Z)

```

1         """Configuration loader and saver.
2
3         Provides a single place to load validated AppConfig with built-in defaults
4         merged with optional file overrides (config.json).
5
6         This replaces scattered json.load + .get() patterns throughout the codebase.
7         """
8
9         from __future__ import annotations
10
11         import json
12         import logging
13         import os
14         from pathlib import Path
15         from typing import Any
16
17         from pydantic import BaseModel
18
19         from backend.utils.app_home import default_config_path
20
21         from .models import AppConfig
22         from .presets import (
23             COMPUTE_TARGET_FOR_PROFILE,
24             PROFILE_PRESETS,
25             is_known_profile,
26             preset_for,
27             suggest_profile,
28         )
29
30         logger = logging.getLogger(__name__)
31
32         # Config path resolution now lives in ``backend.utils.app_home.default_config_path()`` ?
it honours

```

```

33     # ``RALLY_APP_HOME`` (PACKAGING_PLAN.md P0a) and equals ``Path("config.json")`` (CWD-
relative) when
34     # the env is unset, exactly the pre-P0a value. (The old module-level
``DEFAULT_CONFIG_PATH`` constant
35     # was dropped: it had no external consumers and a frozen-at-import snapshot would
silently diverge
36     # from a later RALLY_APP_HOME.)
37
38
39     def _deep_merge(base: dict[str, Any], override: dict[str, Any]) -> dict[str, Any]:
40         """Recursively overlay ``override`` onto ``base`` (override wins; dicts merge,
scalars
41         replace). Used ONLY for the profile-preset precedence path so a config.json sub-key
42         overrides a preset sub-key while sibling preset/default sub-keys survive ? the no-
profile
43         path keeps the original shallow ``{**defaults, **file_data}`` merge unchanged."""
44         out = dict(base)
45         for key, val in override.items():
46             if isinstance(val, dict) and isinstance(out.get(key), dict):
47                 out[key] = _deep_merge(out[key], val)
48             else:
49                 out[key] = val
50         return out
51
52
53     def _resolve_explicit_profile(file_data: dict[str, Any]) -> str:
54         """The EXPLICITLY chosen deployment profile, env winning over config.json. ``""`` if
none
55         is set. Auto-detect is deliberately NOT consulted here ? it only suggests (D15), so
a bare
56         environment never silently selects a profile (and never silently spends on cloud).
57
58         An *unrecognised* ``DEPLOYMENT_PROFILE`` env value warns and FALLS THROUGH to the
file's
59         profile (rather than suppressing it) ? so an env typo can't leave the recorded
profile
60         asserting a preset that was never applied. A bad value in config.json is left to
surface as
61         a hard, typed-Literal error downstream (config-file correctness)."""
62         env_profile = os.environ.get("DEPLOYMENT_PROFILE")
63         if env_profile and env_profile.strip():
64             ep = env_profile.strip()
65             if is_known_profile(ep):
66                 return ep
67             logger.warning(
68                 "[CONFIG] unrecognized DEPLOYMENT_PROFILE env value %r (valid: %s); ignoring
it "
69                 "and falling back to config.json deployment.profile.", ep, ",
".join(PROFILE_PRESETS),
70             )
71             # fall through to the file's profile (env typo must not suppress a valid file
choice)
72             dep = file_data.get("deployment")
73             if isinstance(dep, dict):
74                 prof = dep.get("profile")
75                 if isinstance(prof, str) and prof.strip():
76                     return prof.strip()
77             return ""
78
79
80     def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str =
"") -> list[str]:
81         """Dotted paths in `data` that the typed config will not honor.
82
83         Two silent failure modes exist without this check: a typo'd TOP-LEVEL key is

```

```

84     kept by AppConfig's ``extra="allow"`` but never read by typed code, and a
85     typo'd key INSIDE a section is silently dropped by Pydantic. Either way the
86     user's intent is lost ? collect every such key so load_config can warn.
87     """
88     unknown: list[str] = []
89     fields = model_cls.model_fields
90     for key, val in data.items():
91         if key not in fields:
92             unknown.append(prefix + key)
93             continue
94         ann = fields[key].annotation
95         if isinstance(val, dict) and isinstance(ann, type) and issubclass(ann,
BaseModel):
96             unknown.extend(_unknown_key_paths(val, ann, f"{prefix}{key}."))
97     return unknown
98
99
100 def _get_builtin_defaults() -> dict[str, Any]:
101     """Return a plain dict of the current built-in defaults.
102
103     The AppConfig model is the single source of truth for defaults.
104     """
105     return AppConfig().model_dump(mode="json")
106
107
108 def load_config(path: str | Path | None = None) -> AppConfig:
109     """Load configuration with the following precedence:
110
111     1. Built-in defaults defined in the Pydantic models.
112     2. Values from the JSON file (if present and readable).
113     3. (Future) environment / CLI overrides.
114
115     Missing file or unreadable file ? returns a fully defaulted AppConfig.
116     Unknown keys in the file are tolerated (extra="allow" on the model).
117     """
118     cfg_path = Path(path) if path is not None else default_config_path()
119
120     file_data: dict[str, Any] = {}
121     if cfg_path.exists():
122         try:
123             with open(cfg_path, "r", encoding="utf-8") as f:
124                 file_data = json.load(f)
125         except Exception as exc:
126             # Non-fatal: continue with defaults + whatever we could parse.
127             # In production we might log here.
128             logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
data.")
129
130     # A config.json that is valid JSON but not an OBJECT (e.g. `[ ]`, `42`, `"x"`,
`null`) would
131     # otherwise crash startup: a truthy non-mapping hits `.items()` in
_unknown_key_paths, and any
132     # non-mapping breaks the `**defaults, **file_data` unpack. Degrade to defaults +
warn, per the
133     # loader's "bad input is non-fatal" contract.
134     if not isinstance(file_data, dict):
135         logger.warning("%s is not a JSON object (got %s); ignoring it and using
defaults.",
136                        cfg_path, type(file_data).__name__)
137     file_data = {}
138
139     if file_data:
140         unknown = _unknown_key_paths(file_data, AppConfig)
141         if unknown:
142             logger.warning(

```



```

143         "[CONFIG WARNING] %s contains keys the typed config does not "
144         "recognize (typo'd or removed knobs ? top-level extras are kept "
145         "but unread; unknown section keys are silently dropped): %s",
146         cfg_path, ", ".join(sorted(unknown)),
147     )
148
149     # Precedence: built-in defaults < profile preset < config.json (< env/--set, applied
at
150     # consumption sites downstream). A *deployment profile* is opt-in ? it is applied
ONLY when
151     # explicitly selected (DEPLOYMENT_PROFILE env or config.json deployment.profile).
With no
152     # profile the merge is byte-identical to the pre-M1 loader, so the default path is
unchanged.
153     defaults = _get_builtin_defaults()
154     profile = _resolve_explicit_profile(file_data)
155
156     if profile and not is_known_profile(profile):
157         # Only an unrecognised value from config.json reaches here (env typos already
fell back
158         # in _resolve_explicit_profile). Warn + apply no preset; the bad value also hits
the
159         # typed Literal at model_validate below, a hard clear error ? loud, never
silent.
160         logger.warning(
161             "[CONFIG] unrecognized deployment profile %r (valid: %s); applying no
preset.",
162             profile, ", ".join(PROFILE_PRESETS),
163         )
164         profile = ""
165
166     if profile:
167         merged = _deep_merge(_deep_merge(defaults, preset_for(profile)), file_data)
168         # Record the profile actually applied ? the env may have chosen it over
config.json,
169         # so re-stamp it onto the merged deployment section to keep the record honest.
170         merged.setdefault("deployment", {})
171         if isinstance(merged["deployment"], dict):
172             merged["deployment"]["profile"] = profile
173             ct = merged["deployment"].get("compute_target")
174             canonical = COMPUTE_TARGET_FOR_PROFILE.get(profile)
175             if not ct and canonical:
176                 # An active profile with no explicit compute_target (e.g. config.json
blanked it
177                 # to "") gets its canonical target back ? the record never silently lies
about an
178                 # active profile's compute_target (it stays meaningful for the M2+
ComputeBackend).
179                 merged["deployment"]["compute_target"] = canonical
180             elif ct and canonical and ct != canonical:
181                 logger.warning(
182                     "[CONFIG] deployment profile %r normally uses compute_target=%r but
"
183                     "config.json overrides it to %r ? honoring the explicit override.",
184                     profile, canonical, ct,
185                 )
186                 logger.info("[CONFIG] deployment profile ACTIVE: %s (compute_target=%s)",
187                             profile, merged["deployment"].get("compute_target"))
188     else:
189         # No profile ? the original shallow merge (Pydantic re-defaults nested
sections).
190         merged = {**defaults, **file_data}
191         suggestion = suggest_profile() # cheap, env-only (no torch); SUGGESTS only
(D15)
192         if suggestion:

```

```

193         logger.debug(
194             "[CONFIG] no deployment.profile set (manual knobs). Environment suggests
%r ? "
195             "set deployment.profile or DEPLOYMENT_PROFILE to apply (auto-detect
never "
196             "auto-applies ? D15).", suggestion,
197         )
198
199     return AppConfig.model_validate(merged)
200
201
202     def save_default_config(path: str | Path | None = None) -> Path:
203         """Write a fresh default config.json.
204
205         Used by scripts/doctor.py diagnostics and the --setup CLI path.
206         Overwrites if the file exists.
207         """
208         cfg_path = Path(path) if path is not None else default_config_path()
209         cfg = AppConfig()
210         cfg_path.parent.mkdir(parents=True, exist_ok=True)
211         cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
212         return cfg_path
213
214
215     # The shipped batteries-included LIGHT-tier default (PACKAGING_PLAN.md P2b / gate 08).
216     # Deliberately MINIMAL: the `truly-local` PROFILE preset (presets.py) supplies
skip_ai_handoff,
217     # local storage and the compute_target, so this stays the handful of knobs a fresh, non-
technical
218     # user gets ? a torch-free, offline, zero-setup experience:
219     # * default_segementer "motion" ? pure-OpenCV, no torch / GPU / weights / API key
(LIGHT tier);
220     # * profile "truly-local" ? no Gemini handoff, local storage ? never surprise-
bills on
221     # Gemini nor returns 0 rallies on WASB-without-a-key
(08 intent);
222     # * use_gpu false ? honest for a LIGHT (no-GPU) box (motion ignores it
either way).
223     # The first-run wizard (P2c) upgrades this to Gemini once the user supplies a key.
224     LIGHT_DEFAULT_CONFIG: dict[str, Any] = {
225         "sport": "badminton",
226         "deployment": {"profile": "truly-local"},
227         "indexing": {"default_segementer": "motion", "use_gpu": False},
228     }
229
230
231     def write_light_default_config(
232         path: str | Path | None = None, *, overwrite: bool = False
233     ) -> tuple[Path, bool]:
234         """Seed the batteries-included LIGHT default config (`LIGHT_DEFAULT_CONFIG`).
235
236         Returns `(path, written)`. Unlike :func:`save_default_config` (which dumps a FULL
AppConfig
237         and overwrites), this writes the minimal profile-driven LIGHT bundle and ? by
default ? **never
238         overwrites an existing file**, so a user's edits or the first-run wizard's choices
are preserved
239         (idempotent first-run seeding). Pass ``overwrite=True`` to force. Creates parent
dirs.
240         """
241         cfg_path = Path(path) if path is not None else default_config_path()
242         if cfg_path.exists() and not overwrite:
243             return cfg_path, False
244         cfg_path.parent.mkdir(parents=True, exist_ok=True)
245         cfg_path.write_text(json.dumps(LIGHT_DEFAULT_CONFIG, indent=2) + "\n",

```

```

encoding="utf-8")
246         return cfg_path, True
247
248
249     # Backwards-compatible alias used by some older call sites during transition.
250     def get_default_config() -> dict[str, Any]:
251         """Return defaults as a plain dict (for transitional use only)."""
252         return _get_built_in_defaults()
253

```

11. assistant (2026-06-22T01:26:28.285Z)

Now I need to verify several claims. First, the `truly-local` preset ? does it actually supply `skip\_ai\_handoff` and `storage=local`? And critically, I notice the seeded config uses key `indexing` with `default\_segmenter` and `use\_gpu`. Let me check the actual model field names ? the config models may use a different section name (e.g. the loader references `OutputConfig.output\_dir`, `StitchingConfig`, etc.). Let me check the models and presets.

12. user (2026-06-22T01:26:28.972Z)

```

1     """Deployment-profile presets + environment auto-detection (COMPUTE_DECOUPLED_SERVING
M1).
2
3     A *deployment profile* is a single switch that applies a coherent BUNDLE of existing
4     config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one
5     knob at a time (the design in ``docs/COMPUTE_DECOUPLED_SERVING/README.md`` §4.3).
6
7     Precedence (lowest ? highest), realised by the loader::
8
9         built-in model defaults < profile preset < config.json < env < worker --set
10
11     This module is **pure data + pure functions** ? no IO, and (critically) **no torch
import at
12     module load**. The loader applies a preset ONLY when a profile is EXPLICITLY selected
13     (``config.json`` ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); with no
14     profile selected the loader is byte-identical to the pre-M1 behaviour (``profile=""`` ==
15     today). Auto-detect only ever **suggests** a profile ? it never auto-applies one (D15:
cloud
16     spend is never entered silently).
17     """
18
19     from __future__ import annotations
20
21     import copy
22     import os
23     from typing import Any, Mapping, Optional
24
25     # Canonical enum values, defined ONCE here. ``models.DeploymentConfig``'s ``Literal``
mirrors
26     # these and ``tests/test_config_deployment.py`` pins parity so the two can never drift.
27     PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
28     COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")
29
30     # Each profile maps 1:1 to its canonical compute_target. Used to apply the preset and to
warn
31     # when config.json explicitly overrides compute_target into a state inconsistent with
the profile.
32     COMPUTE_TARGET_FOR_PROFILE: dict[str, str] = {
33         "truly-local": "local_gpu",
34         "cloud-serving": "cloud_run",
35         "cpu-mock": "cpu_mock",
36     }
37
38     # A preset is a partial, nested override dict over the model defaults. It touches ONLY
knobs

```

```

39     # that EXIST today ? M1 is pure/additive: the new
``deployment.{profile,compute_target}`` plus
40     # ``indexing.{detector_impl,skip_ai_handoff}`` and ``storage.backend``. Later milestones
extend
41     # these bundles ? ``input_backend`` (M2) and the configurable output/workspace base (M3,
42     # Launch-Report-gated) ? deliberately NOT set here so M1 never pre-empts a gated flip.
43     PROFILE_PRESETS: dict[str, dict[str, Any]] = {
44         "truly-local": {
45             "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
46             # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU
box).
47             # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
48             "indexing": {"skip_ai_handoff": True},
49             "storage": {"backend": "local"},
50         },
51         "cloud-serving": {
52             "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
53             # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor
(D11).
54             "indexing": {"detector_impl": "native", "skip_ai_handoff": False},
55             "storage": {"backend": "gcs"},
56         },
57         "cpu-mock": {
58             "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
59             # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression
profile.
60             "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
61             "storage": {"backend": "local"},
62         },
63     }
64
65
66     def is_known_profile(profile: str) -> bool:
67         """True for a non-empty, recognised profile name."""
68         return bool(profile) and profile in PROFILE_PRESETS
69
70
71     def preset_for(profile: str) -> dict[str, Any]:
72         """Return a deep COPY of the preset bundle for ``profile`` (so callers can't mutate
the
73         module-level table), or ``{}`` for ``""`` / an unknown profile."""
74         return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))
75
76
77     def _truthy(val: Optional[str]) -> bool:
78         """Common env-var truthiness (``1/true/yes/on``, case/space-insensitive)."""
79         if val is None:
80             return False
81         return val.strip().lower() in {"1", "true", "yes", "on"}
82
83
84     def _cuda_available() -> bool:
85         """Best-effort CUDA probe. torch is heavy + optional, so it is imported lazily here
and
86         ONLY when a caller opts into ``probe_cuda``; any import/runtime failure means 'no
GPU'."""
87         try:
88             # Intentionally lazy + local: keeps the config-load path torch-free (torch is
heavy
89             # and optional). Only reached when a caller opts into probe_cuda.
90             import torch
91
92             return bool(torch.cuda.is_available())
93         except Exception:
94             return False

```

```

95
96
97     def probe_cuda_available() -> bool:
98         """Public best-effort CUDA probe (lazy torch import; ``False`` on any failure).
99
100         Exposed for the per-profile startup checks (``backend/config/startup.py``) so a
101         caller
102         that *wants* the GPU heads-up can pass the result in without the startup module
103         importing
104         torch itself. Best-effort by design: if torch lives only in a detector subprocess
105         env
106         (native WASB), this returns ``False`` in the main process ? which is why the startup
107         checks
108         treat a GPU mismatch as a WARNING (the runner healthcheck is the authority), never a
109         hard
110         error that could false-fail."""
111         return _cuda_available()
112
113     def suggest_profile(
114         env: Optional[Mapping[str, str]] = None, *, probe_cuda: bool = False
115     ) -> str:
116         """SUGGEST (never auto-apply ? D15) a deployment profile from the environment.
117
118         Signals, in priority order:
119
120             - ``K_SERVICE`` set (Cloud Run)           ? ``"cloud-serving"``
121             - ``CI`` truthy                           ? ``"cpu-mock"``
122             - ``probe_cuda`` and a CUDA GPU present ? ``"truly-local"``
123             - otherwise                               ? ``""`` (no strong suggestion)
124
125         ``probe_cuda`` defaults to ``False`` so the common config-load path stays cheap and
126         never
127         imports torch; a CLI ``--detect`` or cloud-serving startup that wants the GPU probe
128         passes
129         ``probe_cuda=True``. The result is advisory only: the caller decides whether to act
130         on it.
131
132         """
133         e = os.environ if env is None else env
134         if e.get("K_SERVICE"):
135             return "cloud-serving"
136         if _truthy(e.get("CI")):
137             return "cpu-mock"
138         if probe_cuda and _cuda_available():
139             return "truly-local"
140         return ""

```

13. assistant (2026-06-22T01:26:32.597Z)

The `truly-local` preset confirms `skip\_ai\_handoff: True` and `storage.backend: local`. Now the seeded config uses `"indexing": {"default\_segmenter": "motion", "use\_gpu": False}`. I need to verify these are the real field names on the `IndexingConfig` model. Let me read models.py.

14. user (2026-06-22T01:26:33.541Z)

```

1     """Pydantic models for the application configuration.
2
3     This module defines the canonical, validated, typed configuration for the
4     Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6     The models support graceful migration from old config.json files via extra="allow".
7     """
8
9     from __future__ import annotations

```

```

10
11 from typing import Any, List, Literal, Optional
12
13 from pydantic import BaseModel, Field, field_validator, model_validator
14
15 Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18 class ValidationRelevanceConfig(BaseModel):
19     court_green_lower: list[int] = Field(default=[35, 40, 40])
20     court_green_upper: list[int] = Field(default=[85, 255, 255])
21     court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24 class ValidationConfig(BaseModel):
25     min_resolution_width: int = 1280
26     min_resolution_height: int = 720
27     min_fps: float = 29.9
28     min_blur_threshold: float = 20.0
29     max_scene_cuts_per_minute: float = 0.5
30     relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)
31
32
33 class TrackNetConfig(BaseModel):
34     wsl_distro: str = "Ubuntu"
35     repo_dir: str = "~/models/TrackNetV4"
36     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37     conda_env: str = "TrackNetV4"
38     weights_path: str = ""
39     queue_length: int = 5
40     stage_in_wsl: bool = True
41     wsl_stage_dir: str = "~/clips"
42     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43     velocity_threshold: float = 0.02
44     # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45     # decoded frame cache) are deleted automatically ? they are pure intermediates,
46     # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47     # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48     keep_frames: bool = False
49     # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50     # 0 = disabled.
51     min_free_gb: float = 0.0
52
53
54 class WasbIndexConfig(BaseModel):
55     """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57     Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58     these knobs actually take effect ? previously this whole block was silently dropped
59     because nested config sections defaulted to extra='ignore'.
60     """
61     wsl_distro: str = "Ubuntu"
62     repo_dir: str = "~/models/WASB-SBDT"
63     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64     conda_env: str = "wasb"
65     weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
66     sport: str = "badminton"
67     wsl_stage_dir: str = "~/clips"
68     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69     velocity_threshold: float = 0.02
70     # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
71     # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.

```

```

72         # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4
CPU fallback
73         # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-
CPU downgrade).
74         device: str = "cuda"          # auto | cuda | mps | cpu
75         python_bin: str = "python"    # the `wasb` conda env python (invoked by path, no
`conda activate`)
76         # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
77         # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
78         # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
79         keep_frames: bool = False
80         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
81         min_free_gb: float = 0.0
82         # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
83         # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
84         # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
85         # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
86         # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).
87         # Tri-state:
88         #     None          = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
89         #                     behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
90         #     True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
91         stream_video: Optional[bool] = None
92
93
94     class FusionConfig(BaseModel):
95         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
96
97         Every default reproduces control behaviour: the fusion segmenter persists the
98         shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
99         true ? the person-cue features, but it does NOT gate the served handoff unless a
100        threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
101        is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
102        supplied ? off-court persons (spectators / adjacent courts) are excluded.
103        """
104        # Which trajectory substrate seeds the windows (shuttle stays primary).
105        substrate: Literal["wasb", "tracknet"] = "wasb"
106        # Windowing velocity threshold for the fusion family (the engine reads
107        # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
108        # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
109        velocity_threshold: float = 0.02
110        # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
111        # enabled ? so the default path never imports torch / touches the GPU.
112        cue_flags: dict[str, bool] = Field(default_factory=dict)
113        # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
114        # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
115        min_crossings: int = Field(default=0, ge=0)
116        # Served Gate B: when the person cue is on, drop windows with median in-court
players

```

```

117     # < this. 0 = OFF.
118     min_players: int = Field(default=0, ge=0)
119     # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
120     court_polygon: Optional[list[list[float]]] = None
121     # Net line for the person two-sided-motion split (person features only ? the shuttle
122     # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
123     net_axis: Literal["x", "y"] = "y"
124     net_position: float = Field(default=0.5, ge=0.0, le=1.0)
125
126
127     class IndexingConfig(BaseModel):
128         default_segmenter: str = "yolo_hybrid"
129         use_gpu: bool = True
130         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
131         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
132         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
133         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
134         # RALLY_DETECTOR_IMPL env var overrides this. See
135         pipeline/detectors/{stub,native_wasb}_runner.py.
136         detector_impl: str = "auto"
137         motion_threshold: float = 0.08
138         min_segment_duration: float = 3.0
139         dead_time_merge_gap: float = 2.0
140         chunk_duration: float = 90.0
141         chunk_overlap: float = 10.0
142         detector_model: str = "fasterrcnn_resnet50_fpn"
143         detector_score_threshold: float = 0.5
144         yolo_velocity_threshold: float = 0.15
145         yolo_frame_skip: int = 3
146         yolo_tracker: str = "bytetrack.yaml"
147         yolo_prefetch_workers: int = 2
148         min_action_window_duration: float = 2.0
149         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
150         # = a window
151         # longer than this many seconds is recursively split at its largest internal
152         inactivity gap (?
153         # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
154         swallow
155         # several rallies + the dead time between them. Never merges, only cuts (recall
156         can't drop).
157         # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
158         Measured n=13
159         # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
160         p=0.0005; cap=6
161         # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
162         0.651?0.161). ***
163         max_window_duration: float = 6.0
164         window_min_split_gap: float = 0.5 # min internal gap (s) that qualifies as a split
165         point
166         # HARD-CAP fallback for W1: when True, an over-long window with NO internal
167         inactivity gap
168         # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
169         is
170         # chopped at its midpoint until ? max_window_duration, making the cap a true upper
171         bound.
172         # Only cuts (recall can't drop). OFF by default until measured/promoted.
173         window_hard_cap: bool = False
174         # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
175         slowly
176         # (picked up / walked) above velocity_thresh, so the window over-extends its END
177         (the measured
178         # [?end] gap vs golden). Trim the post-rally tail back to the last frame whose
179         trailing
180         # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
181         cuts.

```



```

166      # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
167      # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, |?end| 4.96?3.31s (n=13).
The W1 cap
168      # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
169      window_inactivity_end: float = 1.5
170      inactivity_rally_thresh: float = 0.20
171      inactivity_min_density: float = 0.30
172      # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
173      # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
174      # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
175      # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
176      # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
177      # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
178      # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
179      # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
180      window_blob_fallback: bool = False
181      # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
182      # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
183      # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
184      # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
185      window_blob_factor: float = 4.0
186      log_yolo_candidates: bool = True
187      store_yolo_candidates: bool = True
188      ai_handoff_padding: float = 2.0
189      ai_max_workers: int = 5
190      # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
191      # chunks of a high-bitrate (4K) source blow past the provider upload cap
192      # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
193      # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
194      # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
195      # Matches gemini_refine's --clip-scale-width default.
196      ai_chunk_scale_width: int = 1280
197      # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
198      # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
199      # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
200      # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
201      # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
202      skip_ai_handoff: bool = False
203      # Optional debug knob: trim every video to the first N seconds before processing.
204      debug_duration: Optional[float] = None
205
206      tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
207      wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
208      fusion: FusionConfig = Field(default_factory=FusionConfig)
209
210      @field_validator("default_segmenter")
211      @classmethod
212      def segmenter_must_be_registered(cls, v: str) -> str:

```

```

213         # Registry check happens at runtime in main/segmenter selection.
214         # Here we just allow any string for forward compatibility.
215         return v
216
217     @model_validator(mode="after")
218     def _chunk_step_must_be_positive(self) -> "IndexingConfig":
219         # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
220         # step makes `while t < duration: t += step` loop forever, growing memory before
any
221         # GPU work. Reject the bad config at load time instead.
222         if self.chunk_overlap >= self.chunk_duration:
223             raise ValueError(
224                 f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
225                 f"({self.chunk_duration}); otherwise chunking never advances."
226             )
227         return self
228
229
230     class OutputConfig(BaseModel):
231         default_highlights_name: str = "highlights.mp4"
232         db_name: str = "sports_indexer.db"
233         # Directory where the DB, highlight reels, and processed clips are written.
234         # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
235         output_dir: str = "output"
236
237
238     class WatermarkConfig(BaseModel):
239         """Branding overlay burned into each highlight clip during the per-clip re-encode
240         (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
241         (under `stitching.watermark` in config.json) to turn it off. The text is fully
242         configurable. The concat stage stays stream-copy (-c copy)."""
243         enabled: bool = True
244         text: str = "Generated by Khelsutra.guru"
245         logo_path: str = "" # optional transparent PNG overlay; "" = no logo
246         font_path: str = "" # optional .ttf; "" = use the fontconfig `font` family
below
247         font: str = "Sans" # fontconfig family used when font_path is empty
248         font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5) # text height as a
fraction of frame height
249         opacity: float = Field(default=0.85, ge=0.0, le=1.0)
250         box: bool = True # faint backing box behind the text for legibility
251         margin: int = Field(default=24, ge=0) # px inset from the chosen corner
252         position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"
253
254
255     class StitchingConfig(BaseModel):
256         begin_padding: float = 0.5
257         end_padding: float = 0.5
258         use_cache: bool = False
259         min_shot_count: int = 1
260         # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
261         # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
262         # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
263         # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
264         # local detector over-fires and its false positives are mostly SHORT (motion blips,
265         # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
266         # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.

```

```

267     min_rally_duration: float = Field(default=0.0, ge=0.0)
268     watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
269
270
271     class LoggingConfig(BaseModel):
272         """Logging knobs for the run endpoints (see backend/utils/logging_setup.py)."""
273         # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
274         # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
275         # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
276         # them to WARNING; set true to restore full chatter for request-flow debugging.
277         verbose_http: bool = False
278
279
280     class SecurityConfig(BaseModel):
281         # When set, /api/process video_path must resolve under this directory (hosted/tenant
282         # mode). Default None preserves the single-user local 'any local file' workflow.
283         allowed_video_root: Optional[str] = None
284
285         # --- Chunked upload (B2, PR-2) ---
286         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
287         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
288         # contain upload_dir or /api/process will reject the uploaded paths.
289         upload_dir: str = "output/uploads"
290         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
291         # free-tier edge proxies cap request bodies there (HOSTING_PLAN $2).
292         max_upload_mb: int = 4096
293         max_part_mb: int = 95
294         allowed_upload_extensions: List[str] = ["mp4", "mov"]
295
296         # --- M9 edge auth (Cloudflare Access, D9) ---
297         # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local
single-user
298         # path is unchanged). When True, /api/process requires + verifies the `Cf-Access-
Jwt-Assertion`
299         # header against the team JWKS (so a direct hit to the Cloud Run URL can't spoof the
identity
300         # header) and tags the job with the verified user_id (owner_id). The team domain
(e.g.
301         # ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is
the CF Access
302         # application's audience tag. NO secrets here ? these are public identifiers.
303         edge_auth_enabled: bool = False
304         cf_team_domain: str = "" # e.g. https://<team>.cloudflareaccess.com (issuer +
/certs JWKS)
305         cf_access_aud: str = "" # the CF Access application AUD tag the token must
carry
306         # (CORS allow-origins is set via the RALLY_CORS_ALLOW_ORIGINS env var ? read at
import in
307         # backend/main.py so importing the app does no filesystem I/O; default [""].
Credentials stay
308         # OFF ? Cf-Access is a header, not a cookie.)
309
310
311     class StorageConfig(BaseModel):
312         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
313         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
314         Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
315         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
316         (boto3/google auth chains supply credentials). See ``backend/storage/``.
317         backend: Literal["local", "s3", "gcs", "gdrive"] = "local"

```

```

319         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
320         output_root: str = ""
321         # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
322         # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
323         # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
324         # input name is resolved against ``input_root`` (e.g.
``input_root="gs://bucket/videos"`` + "x.mp4"
325         # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
326         # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
input
327         # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
328         # scratch dir before processing (see ``backend.storage.acquire_local_input``).
329         input_backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
330         input_root: str = ""
331         # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
332         # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>]/<video_id>/...
333         # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
334         # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
335         # the legacy flat layout is used unchanged.
336         single_folder_per_video: bool = False
337         # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
338         workspace_root: str = ""
339         # Cloud namespace prefix under the root so per-video folders sit tidily beside
340         # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>). "" = root
directly.
341         workspace_prefix: str = "workspaces"
342         local: dict[str, Any] = Field(default_factory=dict)
343         s3: dict[str, Any] = Field(default_factory=dict)
344         gcs: dict[str, Any] = Field(default_factory=dict)
345         gdrive: dict[str, Any] = Field(default_factory=dict)
346
347
348     class DeploymentConfig(BaseModel):
349         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
350         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
§4.3).
351
352         **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
353         behaves exactly as before this section existed. A profile is opt-in (``config.json``
354         ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
355         *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
356         these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
357         (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
358
359         Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
360
361         # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
362         profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
363         # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
364         # (recorded for observability; consumed by the ComputeBackend seam from M2+).
```

```

365         compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
366
367
368     class BillingConfig(BaseModel):
369         """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).
370
371         **DEFAULT-OFF:** ``enabled=False`` ? nothing records cost (the v6 cost columns stay
372         NULL = today's behaviour). When enabled, a job's cost is computed + stored in **USD** via
373         the versioned ``pricebook`` DB table and **displayed in INR** at ``fx_inr_per_usd``. The default
374         pricebook version is the seeded ``placeholder-v0`` (every rate ``0.0`` ? cost ``0``) until the
375         owner inserts a real, calibrated version and points ``pricebook_version`` at it.
376
377         ? Even with ``enabled=True`` + a real pricebook, recorded cost stays **0** until a
378         follow-up wires the worker to emit per-job ``usage`` (gpu/wall/egress/gemini) ? M8 lands the
379         ledger + pricebook + the recording seam; usage emission (the metering hooks) is a separate
380         step."""
381         enabled: bool = False
382         pricebook_version: str = "placeholder-v0" # the seeded $0 version; owner flips to
383         a real one
384         fx_inr_per_usd: float = 83.0 # USD?INR display rate (Bangalore
385         market)
386         margin: float = 0.0 # resale markup (?0); 0 = bill at
387         cost
388
389     class AppConfig(BaseModel):
390         """Root configuration object.
391
392         All runtime configuration should be accessed via an instance of this class
393         (or the loader) rather than raw dict lookups.
394
395         """
396         sport: Sport = "badminton"
397         ai_provider: str = "gemini"
398         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
399         flagship;
400         # the alias never deprecates underneath us. Eval reports record the run date
401         alongside.
402         ai_model: str = "gemini-flash-latest"
403
404         validation: ValidationConfig = Field(default_factory=ValidationConfig)
405         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
406         output: OutputConfig = Field(default_factory=OutputConfig)
407         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
408         security: SecurityConfig = Field(default_factory=SecurityConfig)
409         logging: LoggingConfig = Field(default_factory=LoggingConfig)
410         storage: StorageConfig = Field(default_factory=StorageConfig)
411         deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
412         billing: BillingConfig = Field(default_factory=BillingConfig)
413
414         # Allow extra keys during transition so existing config.json files do not break.
415         model_config = {
416             "extra": "allow",
417             "validate_assignment": True,
418         }
419
420         # Temporary migration shim for legacy dict-style access (config.get("section",
421         {}).get(...)).

```

```

417 # Will be removed once all call sites (validators, segmenters, etc.) move to typed
418 # attribute access / the typed config. A nested *section* is returned as a plain
dict
419 # via model_dump, so legacy chained access keeps working ? returning the Pydantic
420 # section model instead breaks it (a model has no .get), which bit every validator
on
421 # real runs. Scalar leaves are returned as-is.
422 def get(self, key: str, default: Any = None) -> Any:
423     _MISSING = object()
424     val: Any = _MISSING
425     if key in type(self).model_fields or hasattr(self, key):
426         val = getattr(self, key, _MISSING)
427     if val is _MISSING:
428         # Legacy convenience: find a leaf key inside any nested section. Sections
are
429         # discovered dynamically from the model fields, so a future top-level
section
430         # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
431         for fname in type(self).model_fields:
432             section = getattr(self, fname, None)
433             if isinstance(section, BaseModel) and hasattr(section, key):
434                 val = getattr(section, key)
435                 break
436         if val is _MISSING:
437             return default
438         if isinstance(val, BaseModel):
439             return val.model_dump(mode="json")
440         return val
441
442 def __getitem__(self, key: str) -> Any:
443     val = self.get(key)
444     if val is None and key not in type(self).model_fields:
445         # Re-raise like a real dict for unknown top-level
446         raise KeyError(key)
447     return val
448
449
450 # Convenience type alias for callers that want to annotate
451 Config = AppConfig
452

```

15. assistant (2026-06-22T01:26:40.488Z)

The field names `default\_segmenter` and `use\_gpu` are correct on `IndexingConfig` (lines 128-129). And `sport` is a valid top-level field. The seeded config keys all map to real fields.

Now the critical question: does `default\_segmenter="motion"` actually exist as a registered, torch-free segmenter? Let me check the segmenter registry and the startup checks. Let me also verify how the startup check (`enforce\_startup\_checks` / `\_enforce\_startup\_or\_exit`) treats `cuda\_available=None` and the motion segmenter.

16. user (2026-06-22T01:26:46.525Z)

```

__init__.py
__pycache__
_keyhint.py
base.py
fusion_hybrid.py
gemini.py
motion.py
tracknet_hybrid.py
trajectory_hybrid.py
wasb_hybrid.py
yolo_hybrid.py

```

17. assistant (2026-06-22T01:26:49.359Z)

`motion.py` exists. Let me check it's registered as "motion" and is torch-free, plus the startup checks.

18. user (2026-06-22T01:26:49.738Z)

```
1     import cv2
2     import time
3     import random
4     from typing import List, Dict, Any, Tuple
5     import logging
6     from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
7     from backend.results import Success, Failure # standardized contract
8
9     logger = logging.getLogger(__name__)
10
11     class MotionPlaySegmenter(BasePlaySegmenter):
12         @property
13         def name(self) -> str:
14             return "motion"
15
16         @property
17         def description(self) -> str:
18             return "Traditional CV pixel-motion heuristics tracking frame differences."
19
20         def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
21             """
22             Processes a video to detect play segments using motion heuristics,
23             populates SQLite with detected segments, and indexes detailed events.
24             """
25             if not player_pool:
26                 player_pool = ["Avi", "Suresh", "Ramesh", "Deepak"]
27
28             cap = cv2.VideoCapture(video_path)
29             if not cap.isOpened():
30                 logger.error(f"Segmenter could not open video: {video_path}")
31                 return Failure(message=f"Could not open video: {video_path}",
is_recoverable=False)
32
33             final_segments = self._detect_motion_segments(cap, max_frames)
34
35             segments_data = self._populate_db_with_events(video_id, final_segments,
player_pool)
36
37             return Success(value=segments_data)
38
39         def _detect_motion_segments(self, cap: Any, max_frames: int = None) ->
List[Tuple[float, float]]:
40             """Run the pixel-motion heuristic over ``cap`` and return the detected
41             (start, end) play segments.
42
43             Reads/sub-samples frames, builds a per-frame motion signal, smooths it,
44             thresholds it into raw segments, then merges dead-time gaps and filters
45             out segments shorter than ``min_segment_duration``. ``cap`` is released
46             before returning. This is the detection half of ``process_video`` ? it
47             performs no DB writes and no fabrication.
48             """
49             fps = cap.get(cv2.CAP_PROP_FPS)
50             total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
51             duration = total_frames / fps if fps > 0 else 0
52
53             # Sub-sample settings (5 FPS for analysis)
54             analysis_fps = 5.0
```

```

55     frame_interval = max(1, int(fps / analysis_fps))
56
57     idx_cfg = self.config.get("indexing", {})
58     motion_threshold = idx_cfg.get("motion_threshold", 0.08)
59     min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
60     dead_time_merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
61
62     prev_gray = None
63     motion_signals = []
64     timestamps = []
65
66     frame_idx = 0
67     processed_count = 0
68     t_start = time.time()
69     # Emit a progress line roughly every 5% of the video so long runs are
70     # observable instead of silent (a full 1080p match is ~tens of minutes).
71     progress_step = max(1, total_frames // 20) if total_frames > 0 else 0
72     next_progress = progress_step
73     logger.info(f"[motion] start: {total_frames} frames @ {fps:.1f}fps "
74               f"({duration/60:.1f} min), analyzing every {frame_interval}th
frame")
75     while True:
76         # Skip decoding for intermediate frames using cap.grab() for high
performance
77         for _ in range(frame_interval - 1):
78             if not cap.grab():
79                 break
80             frame_idx += 1
81
82         ret, frame = cap.read()
83         if not ret:
84             break
85
86         # Process frame
87         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
88         # Apply Gaussian Blur to smooth noise
89         gray = cv2.GaussianBlur(gray, (21, 21), 0)
90
91         t = frame_idx / fps
92         timestamps.append(t)
93
94         if prev_gray is not None:
95             # Frame absolute difference (highly optimized motion tracking)
96             diff = cv2.absdiff(prev_gray, gray)
97             _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
98
99             # Dilate to fill gaps
100            thresh = cv2.dilate(thresh, None, iterations=2)
101
102            # Motion ratio
103            motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
104            motion_signals.append(motion_ratio)
105        else:
106            motion_signals.append(0.0)
107
108        prev_gray = gray
109        frame_idx += 1
110        processed_count += 1
111
112        if progress_step and frame_idx >= next_progress:
113            pct = 100.0 * frame_idx / total_frames
114            elapsed = time.time() - t_start
115            eta = elapsed / (frame_idx / total_frames) - elapsed if frame_idx else 0
116            logger.info(f"[motion] {pct:5.1f}% ? {frame_idx}/{total_frames} frames,

```



```

"
117                 f"{processed_count} analyzed, {elapsed:.0f}s elapsed,
~{eta:.0f}s left")
118                 next_progress += progress_step
119
120                 if max_frames is not None and processed_count >= max_frames:
121                     logger.debug(f"Reached max_frames limit of {max_frames}. Halting indexer
frame parsing.")
122                     break
123
124                 if frame_idx >= total_frames:
125                     break
126
127                 cap.release()
128
129                 # Smooth signal using a moving average
130                 window_size = 5
131                 smoothed = []
132                 for i in range(len(motion_signals)):
133                     start = max(0, i - window_size // 2)
134                     end = min(len(motion_signals), i + window_size // 2 + 1)
135                     smoothed.append(sum(motion_signals[start:end]) / (end - start))
136
137                 # Detect active play boundaries
138                 in_segment = False
139                 segment_start = None
140                 raw_segments: List[Tuple[float, float]] = []
141
142                 for i, t in enumerate(timestamps):
143                     signal = smoothed[i]
144                     if signal > motion_threshold:
145                         if not in_segment:
146                             in_segment = True
147                             segment_start = t
148                     else:
149                         if in_segment:
150                             in_segment = False
151                             raw_segments.append((segment_start, t))
152
153                 # Handle video ending while in active segment
154                 if in_segment:
155                     raw_segments.append((segment_start, timestamps[-1]))
156
157                 # Post-Processing: 1. Merge segments separated by less than dead_time_merge_gap
158                 merged_segments: List[Tuple[float, float]] = []
159                 if raw_segments:
160                     curr_start, curr_end = raw_segments[0]
161                     for next_start, next_end in raw_segments[1:]:
162                         if next_start - curr_end < dead_time_merge_gap:
163                             # Merge
164                             curr_end = next_end
165                         else:
166                             merged_segments.append((curr_start, curr_end))
167                             curr_start, curr_end = next_start, next_end
168                 merged_segments.append((curr_start, curr_end))
169
170                 # Post-Processing: 2. Filter segments shorter than min_segment_duration
171                 final_segments: List[Tuple[float, float]] = [
172                     r for r in merged_segments if (r[1] - r[0]) >= min_segment_duration
173                 ]
174
175                 return final_segments
176
177                 def _populate_db_with_events(self, video_id: str, final_segments: List[Tuple[float,
float]]],

```

```

178                 player_pool: List[str]) -> List[Dict[str, Any]]:
179         """Persist ``final_segments`` and their fabricated metadata/events to the DB.
180
181         For each detected segment this assigns a random server/receiver, fabricates
182         rich tags, then indexes a serve / 0?3 smash / termination event set ? all
183         with the stdlib ``random`` module (the legacy demo's fabricated baseline).
184         Returns the per-segment dicts handed back to the caller. The ``random``
185         call order here is load-bearing for reproducibility and must not change.
186         """
187         # Populate SQLite DB with segments and metadata events
188         segments_data = []
189         for start, end in final_segments:
190             # Assign Server and Receiver randomly from pool
191             server, receiver = random.sample(player_pool, 2)
192
193             # Formulate dynamic rich tags
194             dur = end - start
195             metadata = {
196                 "shot_count": random.randint(4, 25),
197                 "intensity": "high" if dur > 12 else "medium",
198                 "avg_speed_kmh": round(random.uniform(40, 85), 1)
199             }
200
201             segment_id = self.db.add_play_segment(video_id, start, end, server,
202 receiver, metadata)
203
204             if segment_id:
205                 # Add sub-segment events
206                 # A: Serve Event (always at start)
207                 serve_foul = random.random() < 0.15 # 15% chance of serve foul
208                 self.db.add_event(
209                     segment_id=segment_id,
210                     timestamp=start + 0.5,
211                     event_type="serve",
212                     player=server,
213                     details={"status": "foul" if serve_foul else "valid", "type":
214 "low_serve" if random.random() > 0.5 else "high_serve"}
215                 )
216
217                 # B: Smash Events (1 to 3 random smashes per segment)
218                 smashes_count = random.randint(0, 3)
219                 for _ in range(smashes_count):
220                     smash_time = random.uniform(start + 1.5, end - 1.0)
221                     smasher = random.choice([server, receiver])
222                     self.db.add_event(
223                         segment_id=segment_id,
224                         timestamp=round(smash_time, 2),
225                         event_type="smash",
226                         player=smasher,
227                         details={"speed_kmh": round(random.uniform(120, 250), 1)}
228                     )
229
230                 # C: Segment Termination Event (foul/winner)
231                 foul_types = ["out_of_bounds", "net_touch", "double_hit", "fault"]
232                 is_foul = random.random() > 0.4
233                 error_player = random.choice([server, receiver])
234
235                 self.db.add_event(
236                     segment_id=segment_id,
237                     timestamp=end - 0.2,
238                     event_type="foul" if is_foul else "winner",
239                     player=error_player,
240                     details={"type": random.choice(foul_types)} if is_foul else
241 {"shot_type": "drop" if random.random() > 0.5 else "smash"}
242                 )

```

```

240
241         # Store segment representation to return to caller
242         segments_data.append({
243             "id": segment_id,
244             "start_time": start,
245             "end_time": end,
246             "duration": dur,
247             "server": server,
248             "receiver": receiver,
249             "metadata": metadata
250         })
251
252     return segments_data
253
254     # Register the segmenter
255     segmenter_registry.register(MotionPlaySegmenter)
256

```

19. assistant (2026-06-22T01:26:54.288Z)

`motion` is registered (line 255) and is pure-OpenCV ? only imports `cv2`, `time`, `random`. No torch. Now let me check the startup checks to understand the claim about `cuda\_available=None` not tripping a startup ERROR. Let me read startup.py and `\_enforce\_startup\_or\_exit`.

20. user (2026-06-22T01:26:54.796Z)

```

1     """Startup checks ? per-profile coherence (M5) + exposure-safety posture (M9 hardening).
2
3     A deployment profile (`truly-local` / `cloud-serving` / `cpu-mock`) implies a
coherent
4     runtime. These checks fail/warn fast at process startup ? a loud signal before any
5     (expensive) job runs ? when the active profile is inconsistent with the environment:
6
7     * cloud-serving with a non-cloud storage backend ? a hard error (the one
config bug we
8     can detect with certainty + zero false-positive risk: a cloud profile that can't
reach a
9     bucket can never serve). This is the README §4.3 "loud error before any job".
10    * a GPU mismatch (e.g. `device='cuda'` on a box where CUDA isn't visible, or
`device='auto'`
11    falling back to CPU) ? a warning, never an error: the probe is best-effort
(torch may live
12    only in the detector subprocess env ? see `presets.probe_cuda_available`), and the
detector's
13    own healthcheck is the real authority. A warning surfaces it early without false-
failing.
14    * creds that can't be confirmed (no `GOOGLE_APPLICATION_CREDENTIALS` / not on
Cloud Run) ?
15    a warning: Application Default Credentials can come from the metadata server /
gcloud, which
16    we can't probe offline, so the storage backend's own init is the authority; we just
flag it.
17
18    Exposure-safety posture (server only, `include_exposure=True`). When the engine is
exposed
19    behind an edge gate (the owner flips `security.edge_auth_enabled` on for the
Cloudflare-Access
20    boundary ? the alpha-shareable "tunnel-to-a-box" topology), the gate is only half the
safety story.
21    A half-configured exposure starts looking healthy then leaks / 500s per request, so we
verify the
22    rest of the posture at boot: the `allowed_video_root` path-jail is set (else an
authenticated
23    tenant could read an arbitrary local file ? error), the CF team-domain + AUD are
present (else

```

```

24     no token can verify ? **error**, lifting the current first-request failure to boot), and
the
25     ``auth`` extra is importable (else every request 500s ? **warning**). Only the HTTP
**server** passes
26     ``include_exposure=True``; the worker reads ``--video-uri`` (not untrusted tenant
paths), so it is
27     never "exposed" and stays unaffected.
28
29     **DEFAULT-OFF (both dimensions):** with no profile selected (``profile == ""``) the
profile checks are
30     a strict **no-op** ? exactly the pre-M5 behaviour; with ``edge_auth_enabled=False``
(today's default)
31     the exposure checks are a strict **no-op** too. So nothing changes for today's manual-
knob deployments.
32     ""
33
34     from __future__ import annotations
35
36     import logging
37     import os
38     from dataclasses import dataclass
39     from typing import Any, Mapping, Optional
40
41     LEVEL_ERROR = "error"
42     LEVEL_WARN = "warn"
43
44     # Storage backends that count as "a cloud bucket" for cloud-serving coherence.
45     _CLOUD_STORAGE = ("gcs", "s3")
46     # STRONG env signals that Google ADC might resolve. Deliberately NOT
GOOGLE_CLOUD_PROJECT /
47     # CLOUDSDK_CONFIG ? those are often set without usable creds (a plain project id / a
config dir
48     # with no active login), so they would suppress the heads-up falsely. (s3 creds are
intentionally
49     # left to boto3's default chain ? no parallel heads-up, by design.)
50     _GCP_ENV_HINTS = ("GOOGLE_APPLICATION_CREDENTIALS", "K_SERVICE", "GCE_METADATA_HOST")
51
52
53     @dataclass(frozen=True)
54     class StartupCheck:
55         """One per-profile finding. ``level`` is ``error`` (fatal) or ``warn`` (heads-
up)."""
56
57         level: str
58         code: str          # stable, greppable code (e.g. "STORAGE_INCOHERENT")
59         message: str
60
61
62     class StartupCheckError(RuntimeError):
63         """Raised by ``enforce_startup_checks`` when an active profile fails a FATAL
check."""
64
65         def __init__(self, failures: list[StartupCheck]):
66             self.failures = failures
67             super().__init__(
68                 "deployment profile startup check failed: "
69                 + "; ".join(f"[{c.code}] {c.message}" for c in failures)
70             )
71
72
73     def _creds_discoverable(env: Mapping[str, str]) -> bool:
74         """True if some Google ADC source is hinted by the environment. Best-effort ? a
False here
75         only downgrades to a WARNING, never a hard failure (the storage init is the
authority)."""

```

```

76         return any(env.get(k) for k in _GCP_ENV_HINTS)
77
78
79     def _auth_extra_available() -> bool:
80         """True if PyJWT (the ``auth`` extra) is importable ? probed WITHOUT importing it
(find_spec),
81         so this stays cheap + side-effect-free. Module-level so tests can monkeypatch it
deterministically
82         (whether PyJWT is installed in the test env must not change the WARN's presence)."""
83         try:
84             import importlib.util
85             return importlib.util.find_spec("jwt") is not None
86         except Exception: # pragma: no cover - find_spec only raises on a broken parent
package
87             return False
88
89
90     def _exposure_checks(config: Any, env: Mapping[str, str]) -> list[StartupCheck]:
91         """Exposure-safety posture for an *exposed* HTTP server. Gated on
``security.edge_auth_enabled``
92         (default False ? ``[]`` = strict no-op, today's behaviour). When edge auth is ON ?
the "I am
93         exposing this engine behind the CF-Access gate" signal ? the gate alone isn't
enough: verify the
94         rest of the posture at boot so a half-configured exposure fails fast instead of
leaking / 500-ing
95         per request. Config-only (no IO beyond the ``find_spec`` auth-extra probe); ``env``
is the
96         caller's environment view (so the host-allowlist check is testable)."""
97         sec = getattr(config, "security", None)
98         if not sec or not getattr(sec, "edge_auth_enabled", False):
99             return [] # DEFAULT-OFF: edge auth off ? no posture enforcement (byte-identical
to today).
100
101         out: list[StartupCheck] = []
102
103         # (1) The path-jail is load-bearing once authenticated tenants can hit /api/process:
WITHOUT it,
104         # a tenant can point the engine at an ARBITRARY local file (allowed_video_root unset
= "any local
105         # file", the single-user default). Fatal for an exposed engine.
106         root = getattr(sec, "allowed_video_root", None)
107         if not root or not str(root).strip():
108             out.append(StartupCheck(
109                 LEVEL_ERROR, "EXPOSED_NO_VIDEO_ROOT",
110                 "security.edge_auth_enabled is ON (engine exposed) but
security.allowed_video_root is "
111                 "unset ? an authenticated tenant could make /api/process read an arbitrary
local file. "
112                 "Set allowed_video_root to a jail directory before exposing the engine.",
113             ))
114
115         # (2) Edge auth cannot verify a token without the team domain + AUD. Today that only
fails on the
116         # FIRST request (auth.resolve_tenant); lift it to BOOT so a misconfigured gate never
starts
117         # "healthy" then 401s every request.
118         team = (getattr(sec, "cf_team_domain", "") or "").strip()
119         aud = (getattr(sec, "cf_access_aud", "") or "").strip()
120         if not team or not aud:
121             missing = " + ".join(n for n, v in (("cf_team_domain", team), ("cf_access_aud",
aud)) if not v)
122             out.append(StartupCheck(
123                 LEVEL_ERROR, "EXPOSED_EDGE_AUTH_INCOMPLETE",
124                 f"security.edge_auth_enabled is ON but {missing} is not configured ? the Cf-

```

```

Access JWT "
125         "cannot be verified (every request would fail). Set security.cf_team_domain
"
126         "(e.g. https://<team>.cloudflareaccess.com) and security.cf_access_aud (the
CF Access "
127         "application AUD tag).",
128     ))
129
130     # (3) The auth extra (PyJWT[crypto]) is lazy-imported by
auth.verify_cf_access_token. If it's
131     # missing, every request 500s at verify time ? a boot WARN beats a per-request 500.
Warning, not
132     # fatal: the auth module's own ImportError is the final authority.
133     if not _auth_extra_available():
134         out.append(StartupCheck(
135             LEVEL_WARN, "EXPOSED_AUTH_EXTRA_MISSING",
136             "security.edge_auth_enabled is ON but PyJWT[crypto] is not importable ? Cf-
Access "
137             "verification will fail at request time. Install the 'auth' extra (pip
install -e .[auth]).",
138         ))
139
140     # (4) P0c: an exposed engine binds 0.0.0.0, but the Host-header (DNS-rebinding)
guard defaults to
141     # loopback-only when RALLY_ALLOWED_HOSTS is unset (see main._allowed_hosts_from_env)
? it would
142     # then 400 every request the edge forwards with its public Host. Boot WARN beats a
silent 400-all.
143     if not (env.get("RALLY_ALLOWED_HOSTS", "") or "").strip():
144         out.append(StartupCheck(
145             LEVEL_WARN, "EXPOSED_HOST_ALLOWLIST_LOOPBACK",
146             "security.edge_auth_enabled is ON but RALLY_ALLOWED_HOSTS is unset ? the
Host-header guard "
147             "defaults to loopback-only and will 400 forwarded requests. Set
RALLY_ALLOWED_HOSTS to your "
148             "public host(s) (or '*' when the edge gate validates the host).",
149         ))
150
151     return out
152
153
154     def run_startup_checks(
155         config: Any,
156         *,
157         cuda_available: Optional[bool] = None,
158         env: Optional[Mapping[str, str]] = None,
159         include_exposure: bool = False,
160     ) -> list[StartupCheck]:
161         """Return the startup findings for ``config``.
162
163         ``cuda_available`` is the caller's best-effort GPU probe (``None`` = not probed ?
GPU checks
164         are skipped; the API server passes ``None``, the worker passes
``presets.probe_cuda_available``).
165         ``include_exposure`` adds the exposure-safety posture (``_exposure_checks``) ? the
HTTP **server**
166         passes ``True``; the worker leaves it ``False`` (it is never "exposed"). Both
dimensions are
167         DEFAULT-OFF: no profile ? no profile checks; ``edge_auth_enabled=False`` ? no
exposure checks.
168         Pure: no IO (beyond the ``find_spec`` auth probe), no torch import ? the caller owns
the GPU probe.
169         """
170         e = os.environ if env is None else env
171         checks: list[StartupCheck] = []

```

```

172
173     # Exposure-safety posture ? INDEPENDENT of deployment.profile (a tunnel-to-a-box
exposure often
174     # runs with no profile set), gated on edge_auth_enabled (default-OFF). Server-only.
175     if include_exposure:
176         checks.extend(_exposure_checks(config, e))
177
178     profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
179     if not profile:
180         return checks # no profile ? only the exposure posture (itself off unless edge
auth is on).
181
182     storage_backend = getattr(getattr(config, "storage", None), "backend", "local")
183     indexing = getattr(config, "indexing", None)
184     detector_impl = getattr(indexing, "detector_impl", "auto")
185     device = getattr(getattr(indexing, "wasb", None), "device", "cuda")
186
187     if profile == "truly-local":
188         # truly-local reads bytes from the local FS or a *mounted* Drive ? a cloud
backend here is
189         # an inconsistent (but not fatal) choice worth surfacing.
190         if storage_backend not in ("local", "gdrive"):
191             checks.append(StartupCheck(
192                 LEVEL_WARN, "STORAGE_UNUSUAL",
193                 f"profile 'truly-local' usually uses local/gdrive storage, but
storage_backend='
194                 f'{{storage_backend}}'. Reads will go to a remote backend.",
195             ))
196         # GPU heads-up (warning only ? the detector healthcheck is the authority; the
probe is
197         # best-effort and may be False even when a subprocess detector env has CUDA).
198         if cuda_available is False:
199             if device == "cuda":
200                 checks.append(StartupCheck(
201                     LEVEL_WARN, "CUDA_NOT_VISIBLE",
202                     "device='cuda' but no CUDA GPU is visible to this process. If the
box truly "
203                     "has no GPU the detector healthcheck will hard-fail ? set
indexing.wasb.device="
204                     "'auto' for the CPU fallback (M4).",
205                 ))
206             elif device == "auto":
207                 checks.append(StartupCheck(
208                     LEVEL_WARN, "CPU_FALLBACK",
209                     "device='auto' and no CUDA GPU is visible to THIS process ? auto may
resolve "
210                     "to CPU (much slower). If torch lives only in the detector
subprocess env this "
211                     "can be a false alarm; the runner's DeviceContext/healthcheck is the
authority.",
212                 ))
213
214     elif profile == "cloud-serving":
215         # The ONE fatal coherence check: a cloud profile that can't reach a bucket can't
serve.
216         if storage_backend not in _CLOUD_STORAGE:
217             checks.append(StartupCheck(
218                 LEVEL_ERROR, "STORAGE_INCOHERENT",
219                 f"profile 'cloud-serving' requires a cloud storage backend "
220                 f"({','.join(_CLOUD_STORAGE)}), but storage_backend='{{storage_backend}}'.
"
221                 "A cloud job cannot read/write a local-only backend.",
222             ))
223         elif storage_backend == "gcs" and not _creds_discoverable(e):
224             checks.append(StartupCheck(

```

```

225         LEVEL_WARN, "CREDS_UNCONFIRMED",
226         "no GOOGLE_APPLICATION_CREDENTIALS / Cloud Run env detected; relying on
Application "
227         "Default Credentials. The GCS backend will fail loudly at init if ADC
can't resolve.",
228     ))
229     if device == "cpu":
230         checks.append(StartupCheck(
231             LEVEL_WARN, "CLOUD_CPU_DEVICE",
232             "profile 'cloud-serving' runs on a GPU (L4) but
indexing.wasb.device='cpu' ? "
233             "detection will not use the GPU.",
234         ))
235     elif cuda_available is False and device in ("cuda", "auto"):
236         checks.append(StartupCheck(
237             LEVEL_WARN, "CLOUD_CUDA_NOT_VISIBLE",
238             "profile 'cloud-serving' expects a GPU but no CUDA GPU is visible to
this process.",
239         ))
240
241     elif profile == "cpu-mock":
242         # The mock profile is for CI/regression ? a real detector here defeats the
purpose.
243         if detector_impl != "stub":
244             checks.append(StartupCheck(
245                 LEVEL_WARN, "MOCK_NONSTUB_DETECTOR",
246                 f"profile 'cpu-mock' normally uses detector_impl='stub', but
detector_impl="
247                 f"'{detector_impl}'. A real detector defeats the GPU-free mock.",
248             ))
249
250     return checks
251
252
253 def enforce_startup_checks(
254     config: Any,
255     *,
256     cuda_available: Optional[bool] = None,
257     env: Optional[Mapping[str, str]] = None,
258     raise_on_error: bool = True,
259     include_exposure: bool = False,
260     logger: Optional[logging.Logger] = None,
261 ) -> list[StartupCheck]:
262     """Run the checks, LOG each (errors at ERROR, warnings at WARNING), and ? unless
263     ``raise_on_error=False`` ? raise ``StartupCheckError`` if any FATAL check tripped.
264
265     ``include_exposure`` (server-only) adds the exposure-safety posture. Returns the
full findings
266     list (so a caller can inspect/surface them). A no-op with no logging and no raise
when both
267     dimensions are off (``run_startup_checks`` returns ``[]``)."""
268     log = logger or logging.getLogger(__name__)
269     checks = run_startup_checks(
270         config, cuda_available=cuda_available, env=env,
include_exposure=include_exposure)
271     for c in checks:
272         if c.level == LEVEL_ERROR:
273             log.error("[STARTUP] %s: %s", c.code, c.message)
274         else:
275             log.warning("[STARTUP] %s: %s", c.code, c.message)
276     errors = [c for c in checks if c.level == LEVEL_ERROR]
277     if errors and raise_on_error:
278         raise StartupCheckError(errors)
279     return checks
280

```


21. assistant (2026-06-22T01:27:00.054Z)

Now I understand the startup checks. For `truly-local` profile with `storage=local`, `cuda\_available=None` (server passes None), and `device='cuda'` (the wasb default ? the seeded config doesn't change it):

- The `STORAGE\_UNUSUAL` check: storage is `local`, so no warning.
- The GPU checks (`CUDA\_NOT\_VISIBLE` / `CPU\_FALLBACK`) only fire when `cuda\_available` is False`. Since the server passes `None`, these are skipped entirely.

So no startup ERROR. Good. Let me verify how `\_enforce\_startup\_or\_exit` is called in main.py (whether the `--app` path passes `include\_exposure` and `cuda\_available`).

22. user (2026-06-22T01:27:01.773Z)

```
54:     enforce_startup_checks,
846: def _enforce_startup_or_exit(config) -> None:
849:     the exposed server (`include_exposure=True`, M9 hardening), a half-configured exposure
(edge
855:         enforce_startup_checks(config, logger=logger, include_exposure=True)
955:     _enforce_startup_or_exit(global_config) # M5: fail fast on an incoherent ACTIVE profile
```

23. user (2026-06-22T01:27:04.581Z)

```
846     def _enforce_startup_or_exit(config) -> None:
847         """M5: run the startup checks for the server path and `sys.exit(1)` on a FATAL
finding BEFORE
848         any work ? a fatal incoherent profile (e.g. cloud-serving pointed at local-only
storage) OR, for
849         the exposed server (`include_exposure=True`, M9 hardening), a half-configured
exposure (edge
850         auth on without the `allowed_video_root` jail / CF team+AUD). A strict no-op when
no
851         `deployment.profile` is selected AND edge auth is off (today's behaviour). No GPU
probe here ?
852         the server stays torch-light; the detector healthcheck is the GPU authority.
Extracted so the
853         fail-fast wiring + the default-OFF no-op are unit-testable."""
854         try:
855             enforce_startup_checks(config, logger=logger, include_exposure=True)
856         except StartupCheckError as e:
857             logger.error("[STARTUP] refusing to start: %s", e)
858             sys.exit(1)
859
```

24. assistant (2026-06-22T01:27:10.970Z)

Confirmed: `\_enforce\_startup\_or\_exit` does NOT pass `cuda\_available` (so it defaults to `None` ? GPU checks skipped) and `include\_exposure=True`. With the seeded config:

- `edge\_auth\_enabled=False` (default) ? exposure checks are a no-op.
- profile `truly-local`, storage `local`, `cuda\_available=None` ? no error, no warning that would block.

So the seeded config loads cleanly with no startup ERROR.

Now let me verify `default\_config\_path()` and the app_home resolution, plus the interaction with `--output-dir`. The key concern: the seeding uses `default\_config\_path()` (config.json under app_home), while `--output-dir` affects DB/output location and the lock. These are independent. Let me confirm `default\_config\_path` and `app\_home`.

25. user (2026-06-22T01:27:11.253Z)

```
1     """Resolved application-data HOME for a packaged, batteries-included install.
2
```

```

3 PACKAGING_PLAN.md P0a. Today every piece of mutable state ? ``config.json``, ``.env``,
4 the SQLite DB, ``output/`` ? is resolved RELATIVE TO THE CURRENT WORKING DIRECTORY, and
5 ``scripts/doctor.py`` writes ``config.json`` to the repo root while the server reads it
from
6 the CWD. Launched from an OS app menu (CWD ? repo root) those disagree and the user
silently
7 "loses" their index. This module introduces ONE resolver every state path roots on.
8
9 Design contract ? **DEFAULT IS BYTE-IDENTICAL TO TODAY**:
10 * ``RALLY_APP_HOME`` env set ? that directory is the app home (the packaged launcher
sets it).
11 * ``RALLY_APP_HOME`` unset ? ``Path(".")`` i.e. the CWD, exactly the pre-P0a
behaviour.
12
13 The OS-default location (``%APPDATA%`` / ``~/Library/Application Support`` /
``~/local/share``)
14 is computed by :func:`default_os_app_home` but is **NOT** auto-selected here ? the
launcher (P2)
15 calls it and exports ``RALLY_APP_HOME`` explicitly. Keeping the resolver opt-in means
importing
16 this module has zero behavioural effect on the repo/dev/CI/test paths (no surprise
writes into a
17 real user profile during a test run).
18
19 stdlib-only (os, platform, pathlib) so it is safe to import at ``backend.main`` line 1 ?
before
20 the heavier pydantic-backed ``backend.config`` package ? keeping the early
``load_dotenv`` path light.
21 """
22
23 from __future__ import annotations
24
25 import os
26 import platform
27 from pathlib import Path
28
29 #: Environment variable the packaged launcher sets to pin the app-data home.
30 APP_HOME_ENV = "RALLY_APP_HOME"
31
32 #: OS-app-dir leaf used by :func:`default_os_app_home` (the product brand).
33 APP_DIR_NAME = "Khelsutra"
34
35
36 def app_home() -> Path:
37     """The resolved application-data home.
38
39     ``RALLY_APP_HOME`` (``~`` expanded) when set; otherwise ``Path(".")`` ? the CWD,
byte-identical
40     to the pre-P0a behaviour. Does NOT create the directory (callers create on write).
41     """
42     env = os.environ.get(APP_HOME_ENV)
43     if env and env.strip():
44         return Path(env.strip()).expanduser()
45     return Path(".")
46
47
48 def app_path(*parts: str | os.PathLike[str]) -> Path:
49     """A path under :func:`app_home` (e.g. ``app_path("output")``)."""
50     return app_home().joinpath(*[str(p) for p in parts])
51
52
53 def default_config_path() -> Path:
54     """``<app-home>/config.json`` ? the config file the loader reads and ``--setup``
writes.
55

```

```

56         With ``RALLY_APP_HOME`` unset this is ``Path("config.json")`` (CWD-relative),
unchanged.
57         """
58         return app_home() / "config.json"
59
60
61     def default_env_path() -> Path:
62         """``<app-home>/env`` ? the dotenv file the server prefers (e.g.
``GEMINI_API_KEY``)."""
63         return app_home() / ".env"
64
65
66     def env_file_to_load() -> Path | None:
67         """The ``.env`` the server should load, or ``None`` meaning "use python-dotenv's
DEFAULT
68         (frame-relative walk-up) search" ? exactly the pre-P0a bare ``load_dotenv()``
behaviour.
69
70         Returns the app-home ``.env`` ONLY when ``RALLY_APP_HOME`` is set AND that file
exists. With the
71         env UNSET this returns ``None`` so the caller keeps the historical search and does
NOT newly
72         prefer a stray CWD ``.env`` (bare ``load_dotenv()`` walks up from the caller file,
not the CWD).
73         """
74         home = app_home()
75         if str(home) == ".": # env unset (or pinned to CWD) ? historical load_dotenv()
search
76             return None
77         candidate = home / ".env"
78         return candidate if candidate.exists() else None
79
80
81     def resolve_output_dir(output_dir: str) -> str:
82         """Root a RELATIVE ``output_dir`` under :func:`app_home`; pass an ABSOLUTE one
through.
83
84         With ``RALLY_APP_HOME`` UNSET the input is returned VERBATIM (the default
``"output"`` when
85         empty) ? byte-identical to the pre-P0a code that stored ``--output-dir`` as-is (no
pathlib
86         normalisation of ``./x`` / separators / trailing slash). With it SET, a relative dir
like
87         ``"output"`` becomes ``<app-home>/output`` so the SQLite DB and reels live with the
rest of the
88         app's state; an absolute path ? or a Windows drive-anchored one (``C:\x`` /
``C:x``) ? passes
89         through untouched.
90         """
91         home = app_home()
92         if str(home) == ".": # env unset (or pinned to CWD) ? verbatim, today's behaviour
93             return output_dir or "output"
94         if not output_dir:
95             return str(home / "output")
96         if os.path.isabs(output_dir) or os.path.splitdrive(output_dir)[0]:
97             return output_dir
98         return str(home / output_dir)
99
100
101     def default_os_app_home(app_name: str = APP_DIR_NAME) -> Path:
102         """The OS-conventional per-user app-data dir (for the packaged LAUNCHER to export as
103         ``RALLY_APP_HOME`` ? NOT auto-selected by :func:`app_home`).
104
105         * Windows ? ``%APPDATA%\<app_name>`` (``~/AppData/Roaming/<app_name>`` if unset)
106         * macOS ? ``~/Library/Application Support/<app_name>``

```

```

107      * Linux/other ? ``$XDG_DATA_HOME/<app_name>`` (``~/local/share/<app_name>`` if
unset)
108      ""
109      system = platform.system()
110      if system == "Windows":
111          base = os.environ.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")
112      elif system == "Darwin":
113          base = str(Path.home() / "Library" / "Application Support")
114      else:
115          base = os.environ.get("XDG_DATA_HOME") or str(Path.home() / ".local" / "share")
116      return Path(base) / app_name
117

```

26. assistant (2026-06-22T01:27:17.586Z)

Both the seeding (`write\_light\_default\_config()` ? `default\_config\_path()`) and the loader (`load\_config()` ? `default\_config\_path()`) resolve config.json via the SAME `default\_config\_path()`, which is based on `app\_home()`. Since `RALLY\_APP\_HOME` is read fresh on each call (not cached), and the seeding runs immediately before `load\_app\_config()`, they will read/write the same path. Correct.

The `--output-dir` and lock are independent of config.json's location ? they use `resolve\_output\_dir` and the DB path. No interaction bug there.

Now let me check one subtle ordering concern: the `--setup` path writes a FULL config via `save\_default\_config` to `default\_config\_path()`. The `--app` seeding writes a MINIMAL config to the same path but never overwrites. These are different code paths (different flags) so no conflict. Let me also verify the app launcher flow ? does `--app` internally re-invoke with `--server` (in a way that might double-seed or change args.app)? Let me check the app launcher.

27. user (2026-06-22T01:27:19.129Z)

```

945:     if args.app:
997:     if args.video_path and not args.server and not args.app:
1078:     if args.app or args.server or not args.video_path:
1079:         if args.app:

```

28. user (2026-06-22T01:27:21.877Z)

```

1078     if args.app or args.server or not args.video_path:
1079         if args.app:
1080             # P2: the friendly launcher ? always localhost-only (use --server for
network exposure),
1081             # auto-pick a free port if --port is busy, and open the bundled UI once the
server is up.
1082             bind_host = "127.0.0.1"
1083             port = _resolve_app_port(bind_host, args.port)
1084             if port != args.port:
1085                 print(f"[APP] Port {args.port} is busy ? using {port} instead.")
1086             url = f"http://{bind_host}:{port}/"
1087             # The browser-open health poll sends Host: 127.0.0.1 ? it relies on loopback
staying in
1088             # the Host allowlist (the default; don't set RALLY_ALLOWED_HOSTS to a non-
loopback-only set
1089             # with --app, or the poll 400s and the browser just won't auto-open ? the
server still runs).
1090             print(f"[APP] Starting KhelSutra ? opening {url} in your browser "
1091                   f"(set RALLY_NO_BROWSER=1 to skip). Press Ctrl+C to stop.")
1092             threading.Thread(target=_open_browser_when_ready, args=(url, url +
"api/health"),
1093                             daemon=True).start()
1094         else:
1095             bind_host = _resolve_bind_host(global_config, args.host) # P0c:
config/profile-driven (default 127.0.0.1)

```

```

1096         port = args.port
1097         print(f"[SERVER] Starting local FastAPI server on
http://{bind_host}:{port}...")
1098         if bind_host not in ("127.0.0.1", "localhost", "::1"):
1099             print(f"[SERVER] ? Binding {bind_host} exposes the API on the network ?
ensure an auth "
1100                 f"gate (edge_auth_enabled / a reverse proxy) is in front, and set
"
1101                 f"RALLY_ALLOWED_HOSTS to your public host(s) (the Host-header
guard defaults to "
1102                 f"loopback-only and will 400 forwarded requests otherwise).")
1103         # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
1104         # in-process wake-timer for any parked 'waiting' job died with the old process,
so
1105         # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
1106         _get_executor().submit(_drain_due_jobs)
1107         uvicorn.run(app, host=bind_host, port=port)
1108
1109     if __name__ == "__main__":
1110         # `python -m backend.main` runs THIS file as `__main__`, but `job_worker`/`uploads`
resolve the
1111         # shared mutable state (`db_instance`, `global_config`) via `import backend.main` ?
a SEPARATE
1112         # module object. If `main()` runs here it configures `__main__`'s globals, leaving
1113         # `backend.main.db_instance` = None; the async-job drain then crashes silently on
1114         # `None.claim_due_job()` and every submitted job hangs in 'queued' forever. Dispatch
through
1115         # `backend.main` so `main()` configures the module everyone actually reads. (No
recursion:
1116         # imported as `backend.main`, `__name__` != "__main__", so this block doesn't re-
fire.)
1117         import backend.main as _bm
1118
1119         _bm.main()
1120

```

29. assistant (2026-06-22T01:27:26.892Z)

The `--app` launcher runs `uvicorn.run(app, ...)` in-process ? it does NOT re-invoke `main()` as a subprocess with `--server`. So the seeding runs exactly once per process. No double-seeding, no subprocess that would skip the seeding. Good.

Now let me check the `--app` path interaction with `args.video\_path`. Line 997: `if args.video\_path and not args.server and not args.app:` is the CLI single-shot. With `--app`, that's skipped. Good ? the seeding at 945 happens regardless of video_path when `args.app` is set.

Let me verify the test file to understand what's covered, and run the relevant tests to confirm correctness.

30. user (2026-06-22T01:27:27.198Z)

```

1     """P2b-install (PACKAGING_PLAN.md): batteries-included config seeding +
install/uninstall tooling.
2
3     Covers:
4         * the engine-side ``write_light_default_config`` (gate 08 default = truly-local +
motion), incl.
5         the load-bearing no-overwrite idempotency and the EFFECTIVE config the loader
produces;
6         * the pure logic of ``scripts/install.py`` / ``scripts/uninstall.py`` (shortcut
content, manifest,
7         installer/uninstaller argv, method resolution, OS app-home) ? no real subprocess/IO.
8     """

```

```

9     from __future__ import annotations
10
11     import importlib.util
12     import json
13     from pathlib import Path
14
15     import pytest
16
17     from backend.config import LIGHT_DEFAULT_CONFIG, load_config, write_light_default_config
18
19
20     def _load_script(name: str):
21         """Load a scripts/*.py bootstrap file as a module (it is stdlib-only, safe to
22         exec)."""
23         path = Path(__file__).resolve().parents[1] / "scripts" / name
24         spec = importlib.util.spec_from_file_location(name[:-3], path)
25         assert spec and spec.loader
26         mod = importlib.util.module_from_spec(spec)
27         spec.loader.exec_module(mod)
28         return mod
29
30     # --- engine seeding: write_light_default_config (gate 08)
31     -----
32
33     def test_seed_writes_truly_local_motion(tmp_path):
34         cfg = tmp_path / "sub" / "config.json" # also proves parent-dir creation
35         path, written = write_light_default_config(cfg)
36         assert written is True and path == cfg and cfg.exists()
37         data = json.loads(cfg.read_text(encoding="utf-8"))
38         assert data == LIGHT_DEFAULT_CONFIG
39         assert data["indexing"]["default_segmenter"] == "motion"
40         assert data["indexing"]["use_gpu"] is False
41         assert data["deployment"]["profile"] == "truly-local"
42
43     def test_seed_does_not_overwrite_existing(tmp_path):
44         cfg = tmp_path / "config.json"
45         cfg.write_text('{"sport": "tennis", "indexing": {"default_segmenter": "gemini"}}',
46         encoding="utf-8")
47         path, written = write_light_default_config(cfg)
48         assert written is False and path == cfg
49         # The user's file is preserved untouched (the wizard/edits are never clobbered).
50         assert json.loads(cfg.read_text(encoding="utf-8"))["indexing"]["default_segmenter"]
51         == "gemini"
52
53     def test_seed_overwrite_true_forces(tmp_path):
54         cfg = tmp_path / "config.json"
55         cfg.write_text("{}", encoding="utf-8")
56         _, written = write_light_default_config(cfg, overwrite=True)
57         assert written is True
58         assert json.loads(cfg.read_text(encoding="utf-8")) == LIGHT_DEFAULT_CONFIG
59
60     def test_seeded_config_loads_to_truly_local_motion(tmp_path):
61         """The minimal seeded file, run through the loader, expands via the truly-local
62         PRESET to a
63         coherent torch-free/offline config (motion + no Gemini handoff + local storage + no
64         GPU)."""
65         cfg = tmp_path / "config.json"
66         write_light_default_config(cfg)
67         loaded = load_config(cfg)
68         assert loaded.indexing.default_segmenter == "motion"
69         assert loaded.indexing.use_gpu is False

```

```

68         assert loaded.deployment.profile == "truly-local"
69         assert loaded.indexing.skip_ai_handoff is True           # supplied by the truly-local
preset
70         assert loaded.storage.backend == "local"                # supplied by the truly-local
preset
71
72
73     # --- scripts/install.py pure logic
-----
74
75     @pytest.fixture(scope="module")
76     def install_mod():
77         return _load_script("install.py")
78
79
80     @pytest.fixture(scope="module")
81     def uninstall_mod():
82         return _load_script("uninstall.py")
83
84
85     @pytest.mark.parametrize(
86         "system, fname, executable",
87         [
88             ("Windows", "KhelSutra.bat", False),
89             ("Darwin", "KhelSutra.command", True),
90             ("Linux", "khelsutra.sh", True)],
91     )
92     def test_shortcut_spec_per_os(install_mod, system, fname, executable):
93         home = Path("/home/u/AppHome")
94         name, content, ex = install_mod.shortcut_spec(system, home)
95         assert name == fname and ex is executable
96         # str(home) is OS-native (backslashes on Windows) ? assert against it, not a
97         # hardcoded literal.
98         assert "RALLY_APP_HOME" in content and str(home) in content
99         assert "indexer --app" in content
100
101     def test_resolve_method(install_mod):
102         assert install_mod.resolve_method("auto", uv_present=True) == "uv"
103         assert install_mod.resolve_method("auto", uv_present=False) == "pip"
104         assert install_mod.resolve_method("pip", uv_present=True) == "pip"    # explicit wins
105         assert install_mod.resolve_method("uv", uv_present=False) == "uv"
106
107     def test_install_command_argv(install_mod):
108         assert install_mod.install_command("uv", ".") == ["uv", "tool", "install", "."]
109         pip = install_mod.install_command("pip", "badminton-highlight-indexer")
110         assert pip[1:] == ["-m", "pip", "install", "badminton-highlight-indexer"]
111
112     def test_default_os_app_home_per_os(install_mod):
113         win = install_mod.default_os_app_home(system="Windows", env={"APPDATA":
114             r"C:\Users\u\AppData\Roaming"})
115         assert win.name == "Khelsutra" and "Roaming" in str(win)
116         lin = install_mod.default_os_app_home(system="Linux", env={"XDG_DATA_HOME":
117             "/home/u/.local/share"})
118         assert str(lin) == str(Path("/home/u/.local/share") / "Khelsutra")
119
120     def test_build_manifest_shape(install_mod):
121         home = Path("/home/u/AppHome")
122         m = install_mod.build_manifest(
123             method="uv", app_home=home, shortcut=home / "khelsutra.sh",
124             created=[home / "khelsutra.sh", home / "uninstall_manifest.json"],
125             timestamp="2026-06-22T00:00:00+00:00",
126         )
127         assert m["package"] == "badminton-highlight-indexer"

```

```

126         assert m["install_method"] == "uv"
127         assert str(home / "config.json") in m["user_data"] # user data is tracked
separately (kept on uninstall)
128         assert m["shortcut"] == str(home / "khelsutra.sh")
129         assert m["schema"] == 1
130
131
132     # --- scripts/uninstall.py pure logic
-----
133
134     def test_uninstall_command_argv(uninstall_mod):
135         assert uninstall_mod.uninstall_command("uv", "badminton-highlight-indexer") == \
136             ["uv", "tool", "uninstall", "badminton-highlight-indexer"]
137         pip = uninstall_mod.uninstall_command("pip", "badminton-highlight-indexer")
138         assert pip[1:] == ["-m", "pip", "uninstall", "-y", "badminton-highlight-indexer"]
139
140
141     def test_find_manifest_explicit(uninstall_mod, tmp_path):
142         m = tmp_path / "uninstall_manifest.json"
143         m.write_text("{} ", encoding="utf-8")
144         assert uninstall_mod.find_manifest(str(m)) == m
145         assert uninstall_mod.find_manifest(str(tmp_path / "nope.json")) is None
146
147
148     def test_install_uninstall_roundtrip_manifest(install_mod, uninstall_mod, tmp_path):
149         """The manifest install.py would write is consumable by uninstall.py's
expectations."""
150         home = tmp_path / "AppHome"
151         m = install_mod.build_manifest(
152             method="pip", app_home=home, shortcut=home / "KhelSutra.bat",
153             created=[home / "KhelSutra.bat", home / "uninstall_manifest.json"],
154             timestamp="2026-06-22T00:00:00+00:00",
155         )
156         mp = home / "uninstall_manifest.json"
157         home.mkdir(parents=True)
158         mp.write_text(json.dumps(m), encoding="utf-8")
159         assert uninstall_mod.find_manifest(str(mp)) == mp
160         loaded = json.loads(mp.read_text(encoding="utf-8"))
161         assert uninstall_mod.uninstall_command(loaded["install_method"],
loaded["package"])[0:3] in (
162             ["uv", "tool", "uninstall"], [install_mod.sys.executable, "-m", "pip"],
163         )
164

```

31. assistant (2026-06-22T01:27:30.010Z)

The tests cover the key engine claims. Let me run the engine-relevant tests to verify they actually pass, plus the app launcher test.

32. user (2026-06-22T01:27:42.908Z)

```

..... [100%]
22 passed in 9.74s

```

33. assistant (2026-06-22T01:27:46.893Z)

All pass. Now let me verify the actual end-to-end behavior with a quick runtime check that the seeded config loads and that the motion segmenter is resolvable from the registry at startup. Let me also check how main.py selects the segmenter from `default\_segmenter` to confirm "motion" resolves without a runtime error (e.g., it doesn't require torch import at registry-resolution time).

34. user (2026-06-22T01:27:50.736Z)


```
motion registered: True
all: ['fusion_hybrid', 'gemini', 'motion', 'tracknet_hybrid', 'wasb_hybrid', 'yolo_hybrid']
```

35. user (2026-06-22T01:27:58.624Z)

```
seeded path: C:\Users\avidu\AppData\Local\Temp\tmpkbi0zxwj\config.json written: True under home:
True
segmenter: motion gpu: False profile: truly-local
skip_ai_handoff: True storage: local compute_target: local_gpu
startup checks (server-style): []
second seed written: False
```

36. assistant (2026-06-22T01:28:03.909Z)

End-to-end confirms every claim in the prompt:

- Seeding roots under `RALLY\_APP\_HOME` (app-home) via `default\_config\_path()`.
- Loads to `motion` + `skip\_ai\_handoff=True` + `storage=local` + `use\_gpu=False` + profile `truly-local`.
- Server-style startup checks (`cuda\_available=None`, `include\_exposure=True`, edge auth off) return `[]` ? NO startup error or warning.
- Idempotent: second seed returns `written=False`.

Now let me verify the `--server` and CLI single-shot byte-identity claim ? that they truly don't seed. The seeding is gated only at `if args.app:` (line 945) and there's no other call to `write\_light\_default\_config` in main.py. Let me confirm.

37. user (2026-06-22T01:28:05.199Z)

```
C:\Users\avidu\Projects\bhi-p2b\backend\main.py:51: write_light_default_config, # P2b:
batteries-included first-run config seeding
C:\Users\avidu\Projects\bhi-p2b\backend\main.py:946: seeded_path, seeded =
write_light_default_config()
C:\Users\avidu\Projects\bhi-p2b\backend\config\__init__.py:13: write_light_default_config,
C:\Users\avidu\Projects\bhi-p2b\backend\config\__init__.py:35: "write_light_default_config",
C:\Users\avidu\Projects\bhi-p2b\backend\config\loader.py:231: def write_light_default_config(
```

38. assistant (2026-06-22T01:28:12.333Z)

Confirmed ? only one call site (line 946) inside the `if args.app:` block. `--server` and CLI single-shot never seed ? byte-identical.

Now let me consider edge cases the prompt asks about more deeply. One subtle thing worth checking: the seeded config sets `use\_gpu: False` and profile `truly-local`. The `truly-local` preset's `compute\_target` is `local\_gpu`. Is there a subtle inconsistency between `use\_gpu: False` and `compute\_target: local\_gpu` that could trip anything? The startup check only looks at `indexing.wasb.device` (default `cuda`), not `indexing.use\_gpu`, and only when `cuda\_available` is `False`. With `cuda\_available=None` it's skipped. So no issue for the server path.

But let me check the worker path ? the prompt focuses on `--app` which runs the server in-process. However, jobs are processed by an in-process worker. Does the worker re-run startup checks with `probe\_cuda\_available()` (which could return `False` on a no-GPU box, triggering a warning)? Let me check whether the worker enforces startup checks in a way that could hard-fail on the motion path.

39. user (2026-06-22T01:28:13.676Z)

```
C:\Users\avidu\Projects\bhi-p2b\backend\config\__init__.py:24: from .presets import
PROFILE_PRESETS, probe_cuda_available, suggest_profile
C:\Users\avidu\Projects\bhi-p2b\backend\config\__init__.py:27: StartupCheckError,
C:\Users\avidu\Projects\bhi-p2b\backend\config\__init__.py:28: enforce_startup_checks,
C:\Users\avidu\Projects\bhi-p2b\backend\config\__init__.py:29: run_startup_checks,
C:\Users\avidu\Projects\bhi-p2b\backend\config\__init__.py:46: "probe_cuda_available",
C:\Users\avidu\Projects\bhi-p2b\backend\config\__init__.py:48: "StartupCheckError",
C:\Users\avidu\Projects\bhi-p2b\backend\config\__init__.py:49: "enforce_startup_checks",
```

```

C:\Users\avidu\Projects\bhi-p2b\backend\config\__init__.py:50: "run_startup_checks",
C:\Users\avidu\Projects\bhi-p2b\backend\config\presets.py:97:def probe_cuda_available() -> bool:
C:\Users\avidu\Projects\bhi-p2b\backend\worker\__main__.py:30: StartupCheckError,
C:\Users\avidu\Projects\bhi-p2b\backend\worker\__main__.py:31: enforce_startup_checks,
C:\Users\avidu\Projects\bhi-p2b\backend\worker\__main__.py:33: probe_cuda_available,
C:\Users\avidu\Projects\bhi-p2b\backend\worker\__main__.py:98:def _run_startup_checks(config:
Any) -> bool:
C:\Users\avidu\Projects\bhi-p2b\backend\worker\__main__.py:104: The CUDA probe is gated
behind an active profile: ``probe_cuda_available()`` lazily imports
C:\Users\avidu\Projects\bhi-p2b\backend\worker\__main__.py:109: cuda = probe_cuda_available()
if profile else None # torch-free on the default-OFF path
C:\Users\avidu\Projects\bhi-p2b\backend\worker\__main__.py:111:
enforce_startup_checks(config, cuda_available=cuda, logger=logger)
C:\Users\avidu\Projects\bhi-p2b\backend\worker\__main__.py:113: except StartupCheckError:
C:\Users\avidu\Projects\bhi-p2b\backend\worker\__main__.py:114: return False # already
logged at ERROR by enforce_startup_checks
C:\Users\avidu\Projects\bhi-p2b\backend\worker\__main__.py:160: if not
_run_startup_checks(config):
C:\Users\avidu\Projects\bhi-p2b\backend\worker\__main__.py:347: if not
_run_startup_checks(config):
C:\Users\avidu\Projects\bhi-p2b\backend\config\startup.py:12: only in the detector subprocess
env ? see ``presets.probe_cuda_available``, and the detector's
C:\Users\avidu\Projects\bhi-p2b\backend\config\startup.py:62:class
StartupCheckError(RuntimeError):
C:\Users\avidu\Projects\bhi-p2b\backend\config\startup.py:63: """Raised by
``enforce_startup_checks`` when an active profile fails a FATAL check."""
C:\Users\avidu\Projects\bhi-p2b\backend\config\startup.py:154:def run_startup_checks(
C:\Users\avidu\Projects\bhi-p2b\backend\config\startup.py:164: are skipped; the API server
passes ``None``, the worker passes ``presets.probe_cuda_available()``).
C:\Users\avidu\Projects\bhi-p2b\backend\config\startup.py:253:def enforce_startup_checks(
C:\Users\avidu\Projects\bhi-p2b\backend\config\startup.py:263: ``raise_on_error=False`` ?
raise ``StartupCheckError`` if any FATAL check tripped.
C:\Users\avidu\Projects\bhi-p2b\backend\config\startup.py:267: dimensions are off
(``run_startup_checks`` returns ``[]``)."""
C:\Users\avidu\Projects\bhi-p2b\backend\config\startup.py:269: checks = run_startup_checks(
C:\Users\avidu\Projects\bhi-p2b\backend\config\startup.py:278: raise
StartupCheckError(errors)

```

40. assistant (2026-06-22T01:28:23.589Z)

The worker (`backend/worker/\_\_main\_\_.py`) is a separate process used in the cloud-decoupled path. For the `--app` LIGHT-tier flow, the job worker is in-process (the comment at main.py:967-968 confirms "The job worker is in-process"). The standalone `backend/worker/\_\_main\_\_.py` is the cloud worker, not invoked by `--app`. Even if it were: with profile `truly-local`, `probe\_cuda\_available()` returns `False` on a no-GPU box, and `device='cuda'` (wasb default) would produce a `CUDA\_NOT\_VISIBLE` **WARNING** ? not a fatal error (the only fatal in `truly-local` is none; `STORAGE\_INCOHERENT` is cloud-serving only). So no hard fail regardless.

I've thoroughly verified all five scrutiny points. Let me do one final check on the in-process job execution path ? does processing a video with the `motion` segmenter (default skip_ai_handoff path) require any torch/GPU import that could break? The motion segmenter itself is pure cv2. But let me confirm the segmenter selection path in main.py doesn't import torch eagerly when `default\_segmenter='motion'`. Actually, the registry already imports all segmenter modules at startup (they're registered), and that import succeeded above without torch. Let me confirm the segmenters package `\_\_init\_\_` doesn't eagerly import torch.

41. user (2026-06-22T01:28:27.388Z)

torch imported after segmenter registry load: False

42. assistant (2026-06-22T01:28:36.603Z)

Torch is not eagerly imported. The LIGHT-tier (torch-free) claim holds.

I have thoroughly verified all five scrutiny areas. Every claim checks out:

1. ****Seeding placement/gating**** ? ``write_light_default_config()`` is called at `main.py:945-946` inside ``if args.app:``, BEFORE ``load_app_config()`` at line 954, and it's the only call site. ``--server`` and CLI single-shot never seed ? byte-identical.
2. ****Idempotent + parent-dir creation**** ? `loader.py:242-246`: returns ``(path, False)`` without writing when the file exists and ``overwrite=False``; ``mkdir(parents=True, exist_ok=True)`` creates parents. Verified by tests and runtime.
3. ****Loads to the right effective config without a startup ERROR**** ? verified end-to-end: `motion + skip_ai_handoff=True + storage=local + use_gpu=False + profile truly-local; server-style` checks (``cuda_available=None``, ``include_exposure=True``, edge auth off) return ``[]``.
4. ****No interaction bug**** with ``--output-dir`` / `app_home` / `P0e` lock ? `config.json` path and `output/DB/lock` path are independent resolvers; both config seeding and loading use the same ``default_config_path()``.
5. ****`motion` is a real registered torch-free segmenter**** ? registered as "motion", pure cv2, no torch import.

No real bugs found. Returning an empty findings list.

43. user (2026-06-22T01:28:36.607Z)

Structured output provided successfully

44. assistant (2026-06-22T01:28:48.071Z)

I reviewed PR #310 "P2b-install" for correctness and edge cases of the engine change. All five scrutiny points verified clean ? no real bugs found.

Summary of what I verified (files in ``C:\Users\avidu\Projects\bhi-p2b``):

1. ****Seeding placement/gating**** (``backend/main.py:945-946``) ? ``write_light_default_config()`` runs inside ``if args.app:``, BEFORE ``load_app_config()`` at line 954, and grep confirms it's the only call site. ``--server`` and CLI single-shot never seed ? byte-identical. The ``--app`` launcher runs ``uvicorn.run()`` in-process (line 1107), not a subprocess re-invocation, so seeding fires exactly once.
2. ****Idempotent + parent-dir creation**** (``backend/config/loader.py:241-246``) ? returns ``(path, False)`` without writing when the file exists and ``overwrite=False``; ``mkdir(parents=True, exist_ok=True)`` creates parents. Confirmed by runtime (second seed ? ``written=False``) and tests.
3. ****Loads to the right effective config without a startup ERROR**** ? end-to-end run produced ``motion`` + ``skip_ai_handoff=True`` + ``storage=local`` + ``use_gpu=False`` + profile ``truly-local`` (the ``truly-local`` preset in ``presets.py:44-50`` supplies handoff/storage). Server-style ``enforce_startup_checks(cuda_available=None, include_exposure=True)`` with edge auth off returned ``[]`` ? the GPU checks (``startup.py:198``) only fire on ``cuda_available is False``, and the server passes ``None``; the only fatal check (``STORAGE_INCOHERENT``) is cloud-serving-only.
4. ****No interaction bug**** with ``--output-dir`` / `app_home` / the `P0e` single-instance lock ? `config.json` is resolved by ``default_config_path()`` (same call for both seed and load), while `output/DB/lock` use the independent ``resolve_output_dir`/`db_path``. ``RALLY_APP_HOME`` is read fresh per call, so seed and load agree.
5. ****`motion` is a real registered torch-free segmenter**** (``backend/pipeline/segmenters/motion.py:255``) ? pure-cv2; confirmed ``torch`` is not imported when the segmenter registry loads.

``tests/test_p2b_install.py`` + ``tests/test_app_launcher.py`` (22 tests) pass.

Findings list: empty.